
Computer Systems

Kasun Gunawardana

E-mail: kgg



University of Colombo School of Computing

Combinational Logic Circuits

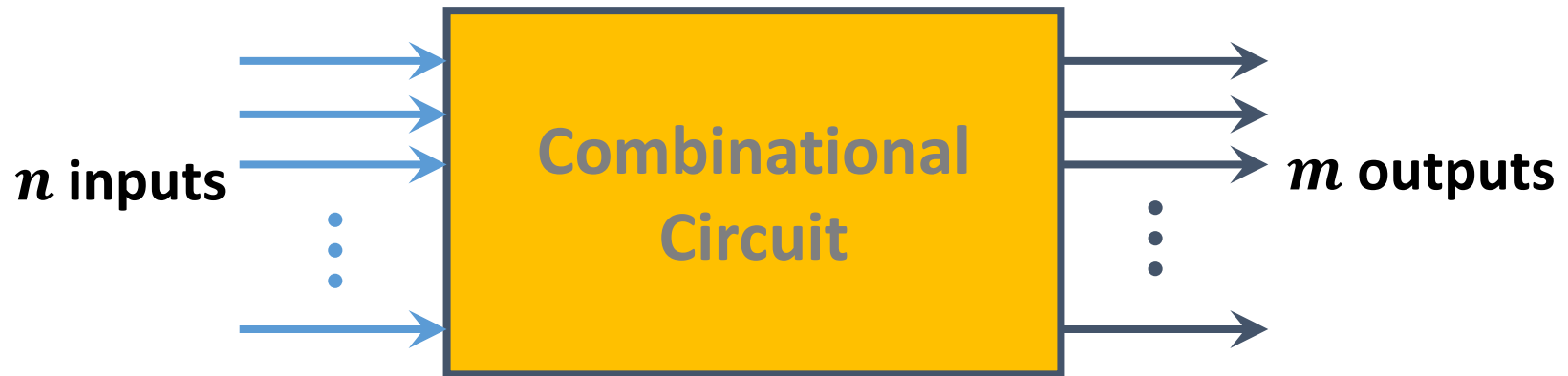
Digital Logic Circuits

- There are two types of digital logic circuits.
 - Combinational Logic Circuits
 - Sequential Logic Circuits

Combinational vs Sequential

- The output of a combinational circuit depends **only** on its current inputs.
 - Purely built upon logic gates
 - No feedback paths, No memory elements
- But, there are times we need circuits that consider not only its present inputs, but also the previous outputs that are stored in storage elements.
- Such circuits are considered as sequential circuits.
- Sequential circuits store previous outputs.

Combinational Circuit



- 2^n possible input patterns
- Circuit can be represented by a truth table
- Circuit can be defined as **m** Boolean function

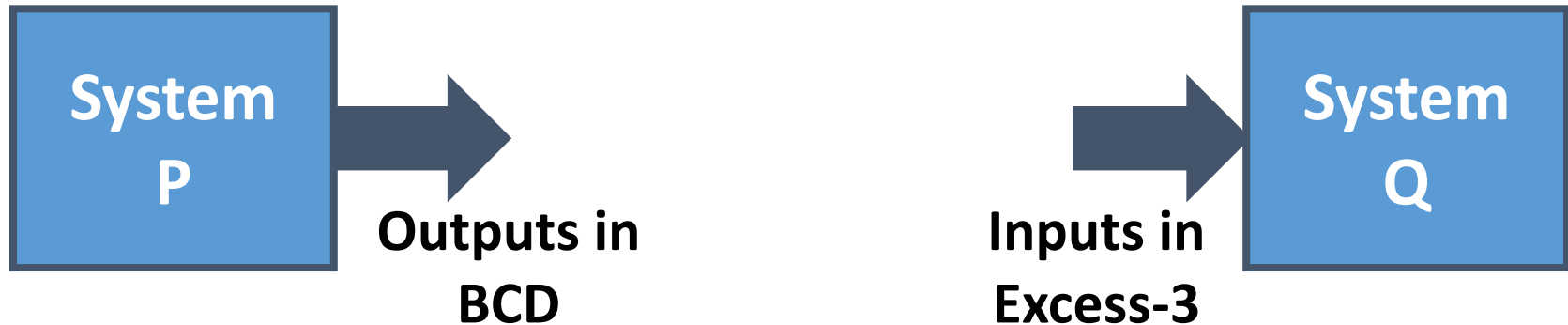
Digital Systems

- There are standard units of combinational circuit in digital systems.
 - Adders, decoders, encoders, multiplexers, etc.
- These are available as integrated circuits (ICs)
 - MSI – Medium Scale Integration circuits
 - VLSI – Very Large Scale Integration circuits

Challenge #1

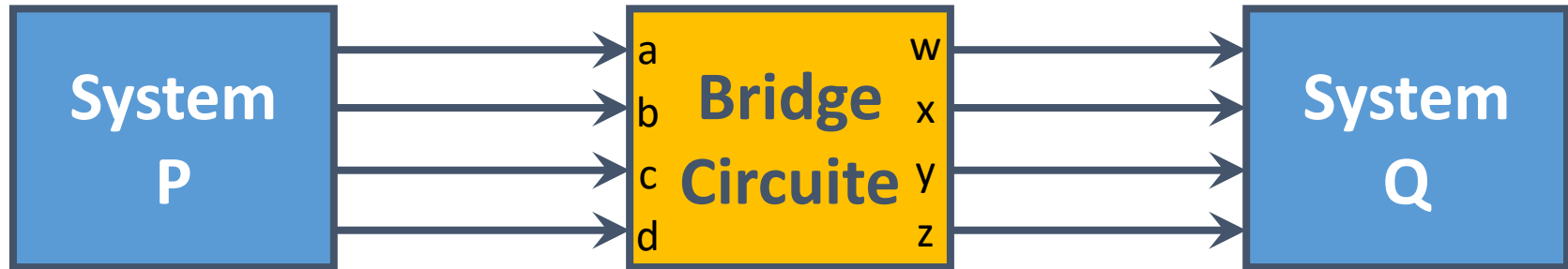
- Assume there are two digital systems, P and Q.
- System P generates 0-9 decimal digits. But digits are encoded in Binary (Binary Coded Decimal).
- System Q takes decimal digit as input, but it should be in Excess-3 encoding.
- Two systems should be connected as P's output will be fed into Q.
- But two systems operate on two different encoding systems.
- Therefore, code translator should be designed in between P and Q to bridge two systems.

Challenge #1: Block Diagram



- P generates 10 different binary patterns.
 - P has 4 output channels
- 10 different Excess-3 patterns should be fed into Q.
 - Q has 4 input channels

Challenge #1: Block Diagram



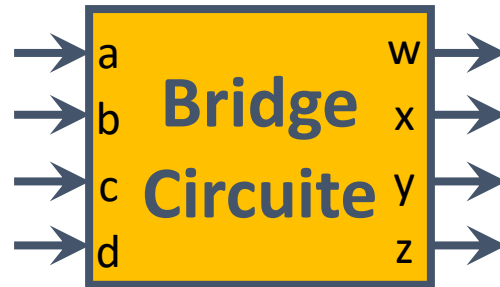
- P generates 10 different binary patterns.
- Therefore we need minimum 4 binary digits for Bridge circuit input.
- Q accepts 10 different excess-3 patterns.
- Therefore we need minimum 4 binary digits for Bridge circuit output.

Conversion Table

Decimal	Binary Coded Decimal				Excess-3			
	a	b	c	d	w	x	y	z
0	0	0	0	0	0	0	1	1
1	0	0	0	1	0	1	0	0
2	0	0	1	0	0	1	0	1
3	0	0	1	1	0	1	1	0
4	0	1	0	0	0	1	1	1
5	0	1	0	1	1	0	0	0
6	0	1	1	0	1	0	0	1
7	0	1	1	1	1	0	1	0
8	1	0	0	0	1	0	1	1
9	1	0	0	1	1	1	0	0

Designing Bridge Circuit

a	b	c	d
0	0	0	0
0	0	0	1
0	0	1	0
0	0	1	1
0	1	0	0
0	1	0	1
0	1	1	0
0	1	1	1
1	0	0	0
1	0	0	1



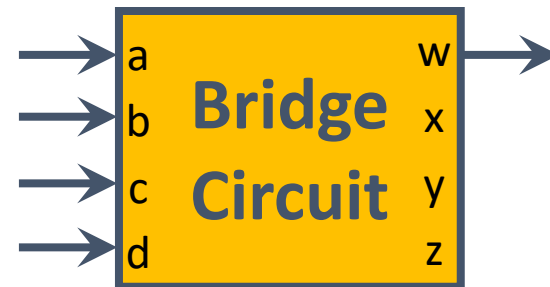
w	x	y	z
0	0	1	1
0	1	0	0
0	1	0	1
0	1	1	0
0	1	1	1
1	0	0	0
1	0	0	1
1	0	1	0
1	0	1	1
1	1	0	0

Bridge Circuit – Design Approach

- The Bridge Circuit has 4 inputs.
- $2^4=16$ input patterns are possible.
- But there are only 10 defined input patterns.
- Remaining patterns are the “Don’t Care Conditions”
- There are 4 outputs for the Bridge Circuit.
- Draw K-Maps for all output variables and derive the simplified Boolean logic expressions.

Output w

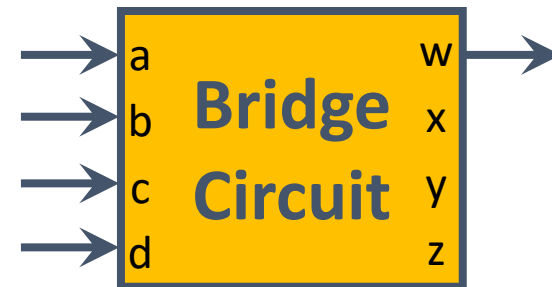
a	b	c	d	w
0	0	0	0	0
0	0	0	1	0
0	0	1	0	0
0	0	1	1	0
0	1	0	0	0
0	1	0	1	1
0	1	1	0	1
0	1	1	1	1
1	0	0	0	1
1	0	0	1	1



		ab			
		00	01	11	10
cd	00	0	0	x	1
	01	0	1	x	1
	11	0	1	x	x
	10	0	1	x	x

Output w

a	b	c	d	w
0	0	0	0	0
0	0	0	1	0
0	0	1	0	0
0	0	1	1	0
0	1	0	0	0
0	1	0	1	1
0	1	1	0	1
0	1	1	1	1
1	0	0	0	1
1	0	0	1	1

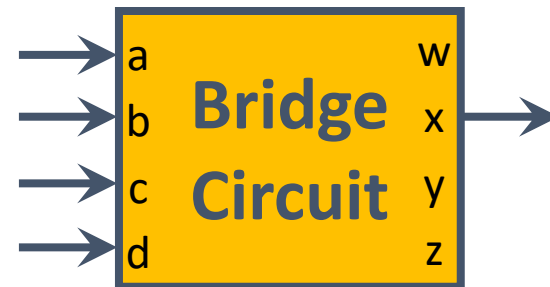


	ab	00	01	11	10
cd					
00		0	0	x	1
01		0	1	x	1
11		0	1	x	x
10		0	1	x	x

$$w = a + bc + bd$$

Output x

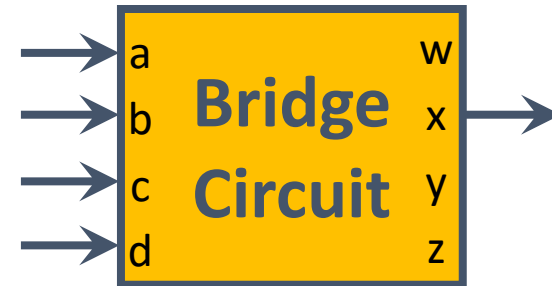
a	b	c	d	x
0	0	0	0	0
0	0	0	1	1
0	0	1	0	1
0	0	1	1	1
0	1	0	0	1
0	1	0	1	0
0	1	1	0	0
0	1	1	1	0
1	0	0	0	0
1	0	0	1	1



ab		00	01	11	10
cd	00	0	1	x	0
	01	1	0	x	1
	11	1	0	x	x
	10	1	0	x	x
	00	0	1	x	0

Output x

a	b	c	d	x
0	0	0	0	0
0	0	0	1	1
0	0	1	0	1
0	0	1	1	1
0	1	0	0	1
0	1	0	1	0
0	1	1	0	0
0	1	1	1	0
1	0	0	0	0
1	0	0	1	1



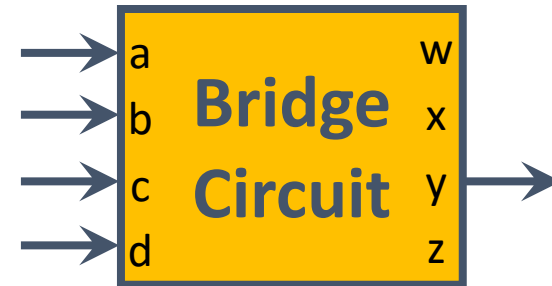
	ab	00	01	11	10
cd					
00		0	1	x	0
01		1	0	x	1
11		1	0	x	x
10		1	0	x	x

The Karnaugh map shows groupings with blue and red lines. Blue lines group the 1s in the first column (cd=01, 11, 10) and the 1s in the second column (cd=00, 01, 11, 10). Red lines group the 1s in the first row (cd=00, 01) and the 1s in the last row (cd=11, 10).

$$x = b'c + b'd + bc'd'$$

Output y

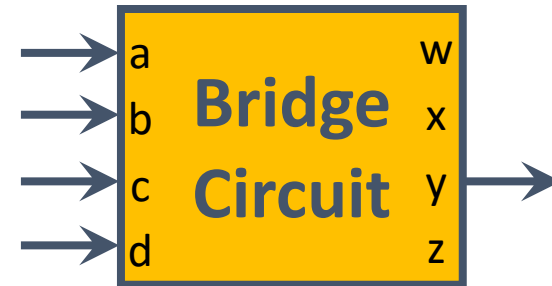
a	b	c	d	y
0	0	0	0	1
0	0	0	1	0
0	0	1	0	0
0	0	1	1	1
0	1	0	0	1
0	1	0	1	0
0	1	1	0	0
0	1	1	1	1
1	0	0	0	1
1	0	0	1	0



		ab			
		00	01	11	10
cd	00	1	1	x	1
	01	0	0	x	0
	11	1	1	x	x
	10	0	0	x	x

Output y

a	b	c	d	y
0	0	0	0	1
0	0	0	1	0
0	0	1	0	0
0	0	1	1	1
0	1	0	0	1
0	1	0	1	0
0	1	1	0	0
0	1	1	1	1
1	0	0	0	1
1	0	0	1	0

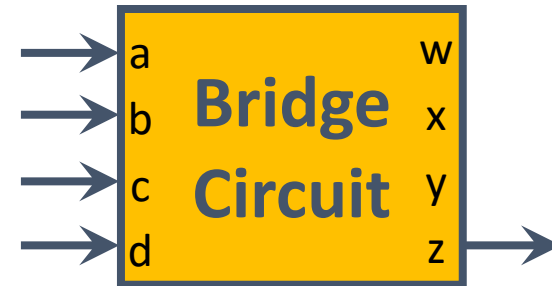


	ab	00	01	11	10
cd	00	1	1	x	1
	01	0	0	x	0
	11	1	1	x	x
	10	0	0	x	x

$$y = c'd' + cd$$

Output z

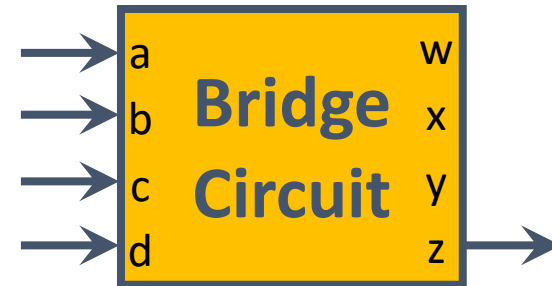
a	b	c	d	z
0	0	0	0	1
0	0	0	1	0
0	0	1	0	1
0	0	1	1	0
0	1	0	0	1
0	1	0	1	0
0	1	1	0	1
0	1	1	1	0
1	0	0	0	1
1	0	0	1	0



ab		00	01	11	10
cd	00	1	1	x	1
	01	0	0	x	0
	11	0	0	x	x
	10	1	1	x	x
	00	1	1	x	1

Output z

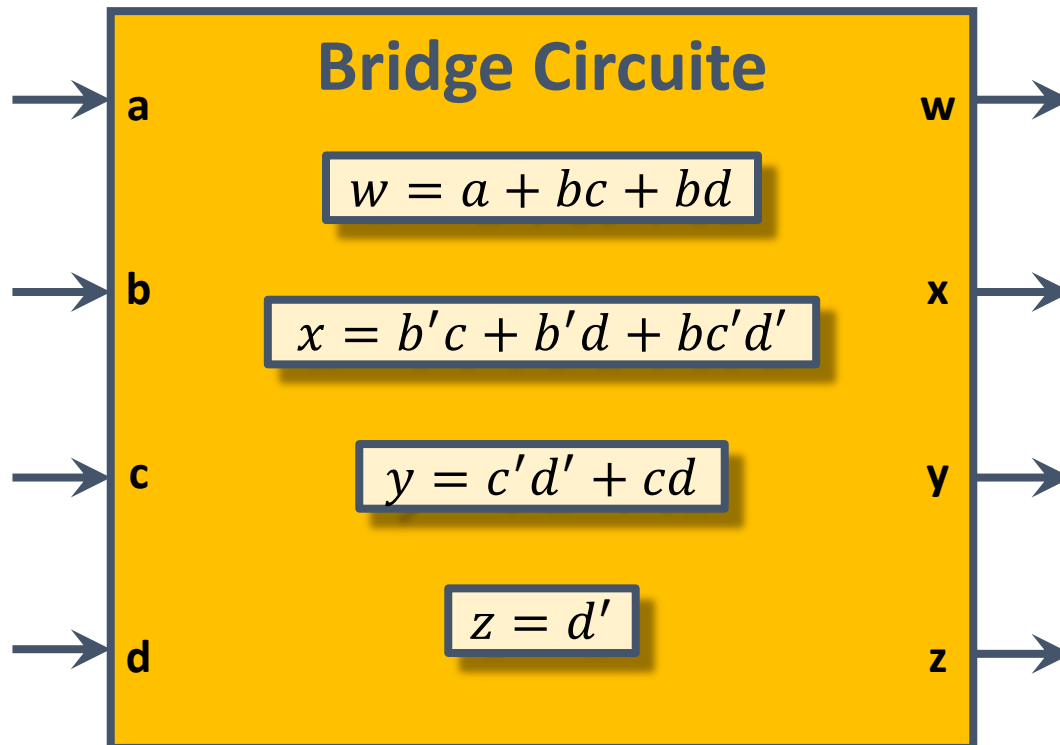
a	b	c	d	z
0	0	0	0	1
0	0	0	1	0
0	0	1	0	1
0	0	1	1	0
0	1	0	0	1
0	1	0	1	0
0	1	1	0	1
0	1	1	1	0
1	0	0	0	1
1	0	0	1	0



	ab	00	01	11	10
cd	00	1	1	x	1
	01	0	0	x	0
	11	0	0	x	x
	10	1	1	x	x

$$z = d'$$

K-Maps for Four w, x, y and z



Next...

Binary Adder

Computer Systems

Kasun Gunawardana

E-mail: kgg



University of Colombo School of Computing

Binary Adders

Binary Adder

- **ADD** is an important operation.
- Most basic version of **ADD** is adding two digits.
- Combinational circuit that adds two bits is called “Half Adder”.
- Half adder cannot accommodate a carry in bit.

Half Adder

- Adds only two bits.
- Four possible cases.

0	0	1	1
0	1	0	1
0	1	1	1 0



- In the final case there is a carry out.
- Therefore, Half Adder circuit has 2 inputs and 2 outputs.

Half Adder Circuit Design

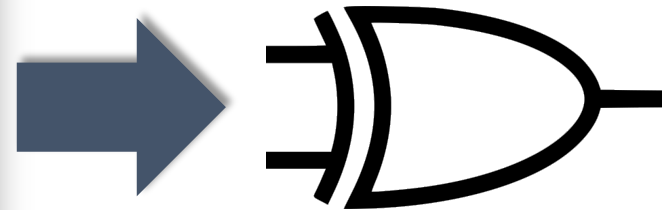
- Let's have the truth table.
- Two inputs, one for each bit
- Two outputs, Sum and Carry
- We can derive logic expressions for two outputs separately.

Inputs		Outputs	
a	b	Sum	Carry Out
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

Half Adder Circuit Design

a	b	Sum
0	0	0
0	1	1
1	0	1
1	1	0

Inputs		Outputs	
a	b	Sum	Carry Out
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

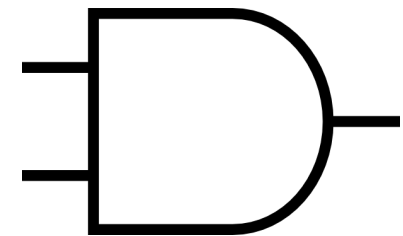
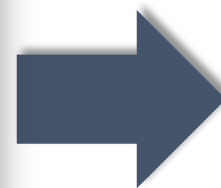


XOR Gate

Half Adder Circuit Design

a	b	Carry Out
0	0	0
0	1	0
1	0	0
1	1	1

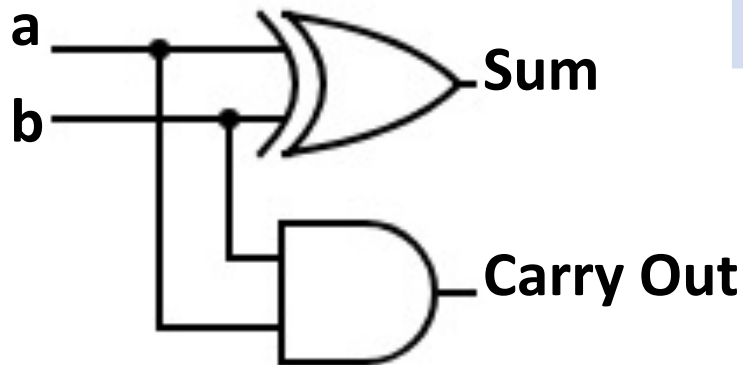
Inputs		Outputs	
a	b	Sum	Carry Out
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1



AND Gate

Half Adder - Circuit

- Half Adder circuit can be designed as below.



Inputs		Outputs	
a	b	Sum	Carry Out
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

Full Adder

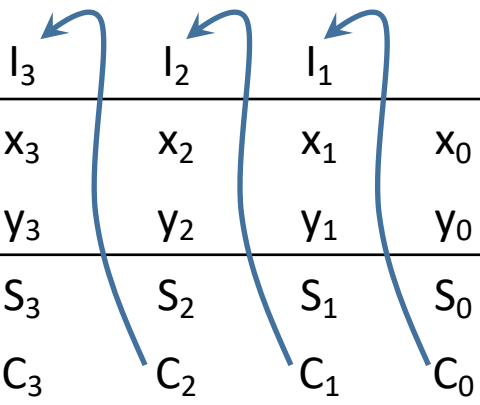
- Half Adder can only add two single bit numbers.
- But digital systems should be able to add two numbers where each number has more than 1 bit.
- Let's assume two 4-bit numbers, x and y ,

Carry In	I_3	I_2	I_1	
Number 1 (x)	x_3	x_2	x_1	x_0
Number 2 (y)	y_3	y_2	y_1	y_0
Sum	S_3	S_2	S_1	S_0
Carry Out	C_3	C_2	C_1	C_0

Full Adder

- Half Adder can only add two single bit numbers.
- But digital systems should be able to add two numbers where each number has more than 1 bit.
- Let's assume two 4-bit numbers, x and y ,

Carry In	I_3	I_2	I_1	
Number 1 (x)	x_3	x_2	x_1	x_0
Number 2 (y)	y_3	y_2	y_1	y_0
Sum	S_3	S_2	S_1	S_0
Carry Out	C_3	C_2	C_1	C_0



The diagram illustrates the carry propagation in a 4-bit adder. Blue arrows show the flow of carry bits from right to left: C_0 from x_0 and y_0 to S_1 ; C_1 from x_1 and y_1 to S_2 ; C_2 from x_2 and y_2 to S_3 ; and C_3 from x_3 and y_3 as the final carry out.

Full Adder

- To be a complete addition unit, a circuit should have,
 - INPUT: Two bits to be added
 - INPUT: A bit represents Carry in
 - OUTPUT: A bit to hold Sum of all inputs
 - OUTPUT: A bit to hold Carry out



Full Adder - Truth Table

Inputs			Outputs	
a	b	C _{in}	Sum	C _{out}
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

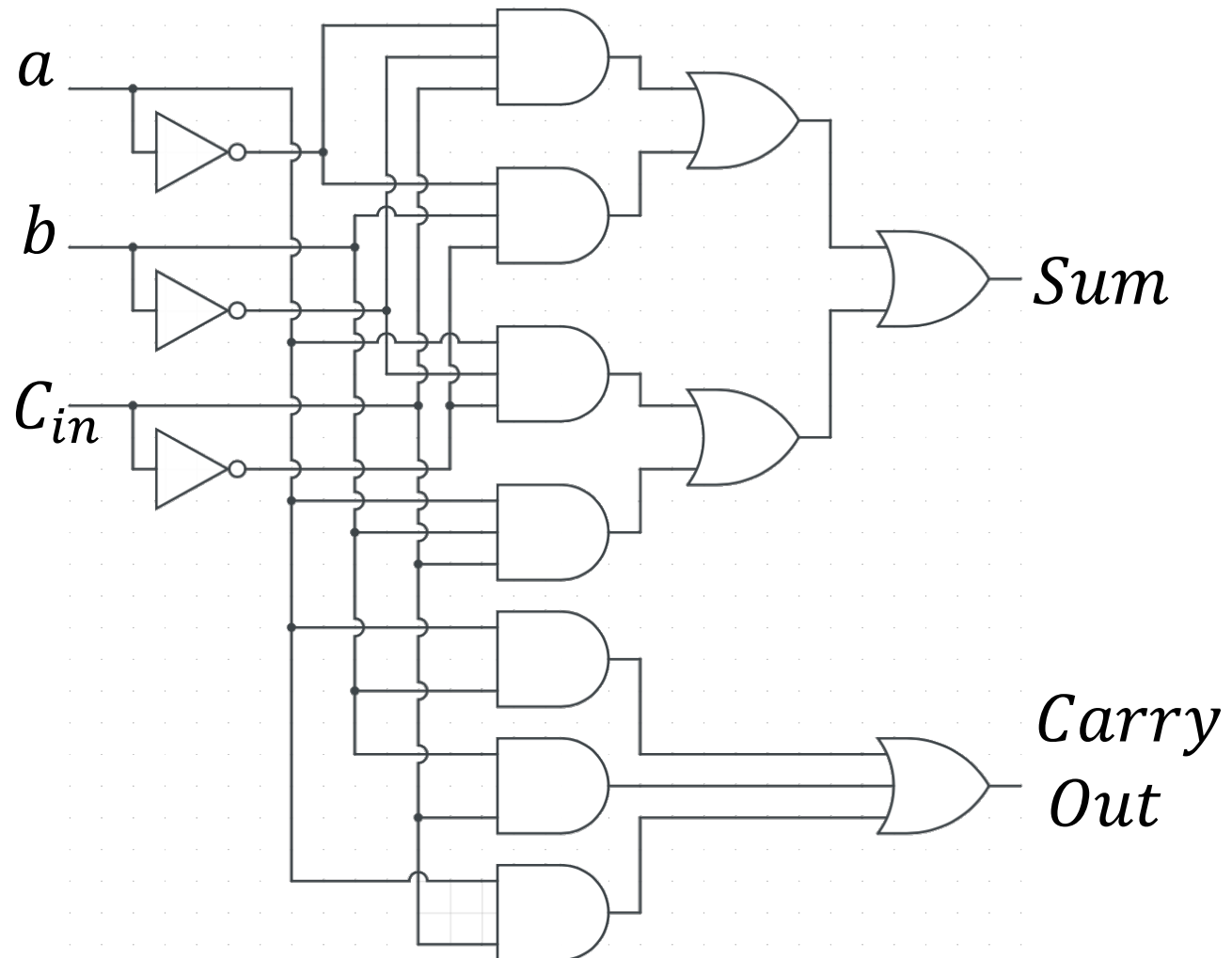
After simplification

$$\begin{aligned} \text{Sum} \\ &= a'b'C_{in} + a'bC'_{in} + ab'C'_{in} + abC_{in} \end{aligned}$$

$$C_{out} = ab + bC_{in} + aC_{in}$$

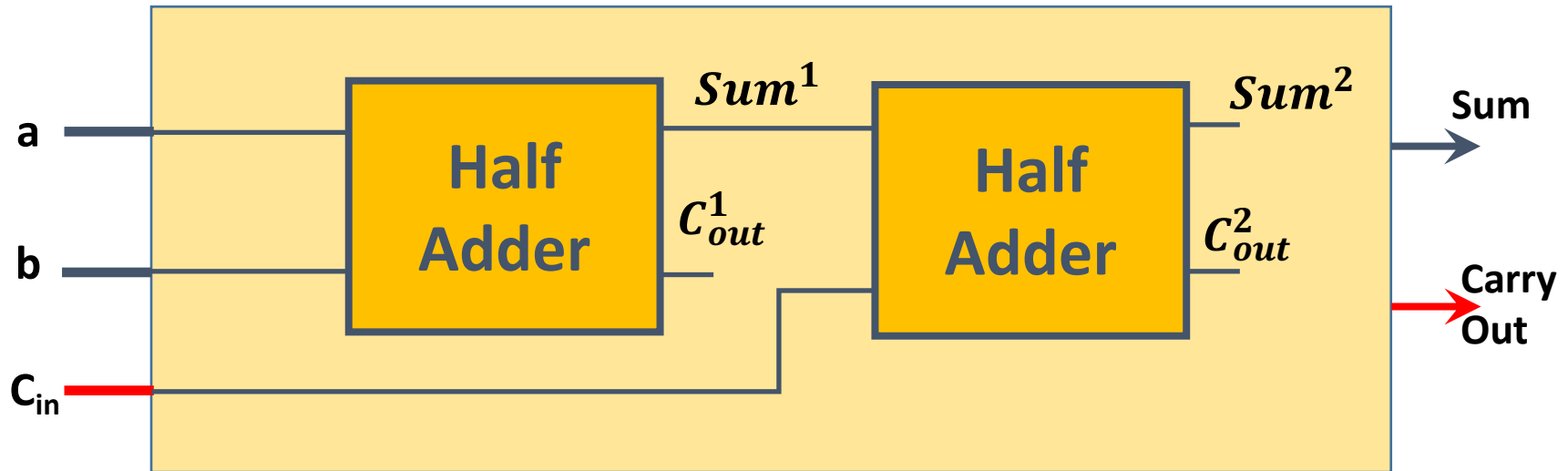
$$Sum = a'b'C_{in} + a'bC_{in}' + ab'C_{in}' + abC_{in}$$

$$C_{out} = ab + bC_{in} + aC_{in}$$



Full Adder – Alternative Approach

- Full adder takes 3 inputs (3 bits).
- Half adder can work with 2 inputs (2 bits)



Sum

a	b	<i>Sum</i>¹	<i>C</i>_{in}	<i>Sum</i>²	Sum (Expected)
0	0	0	0	0	0
0	1	1	0	1	1
1	0	1	0	1	1
1	1	0	0	0	0
0	0	0	1	1	1
0	1	1	1	0	0
1	0	1	1	0	0
1	1	0	1	1	1

Carry Out

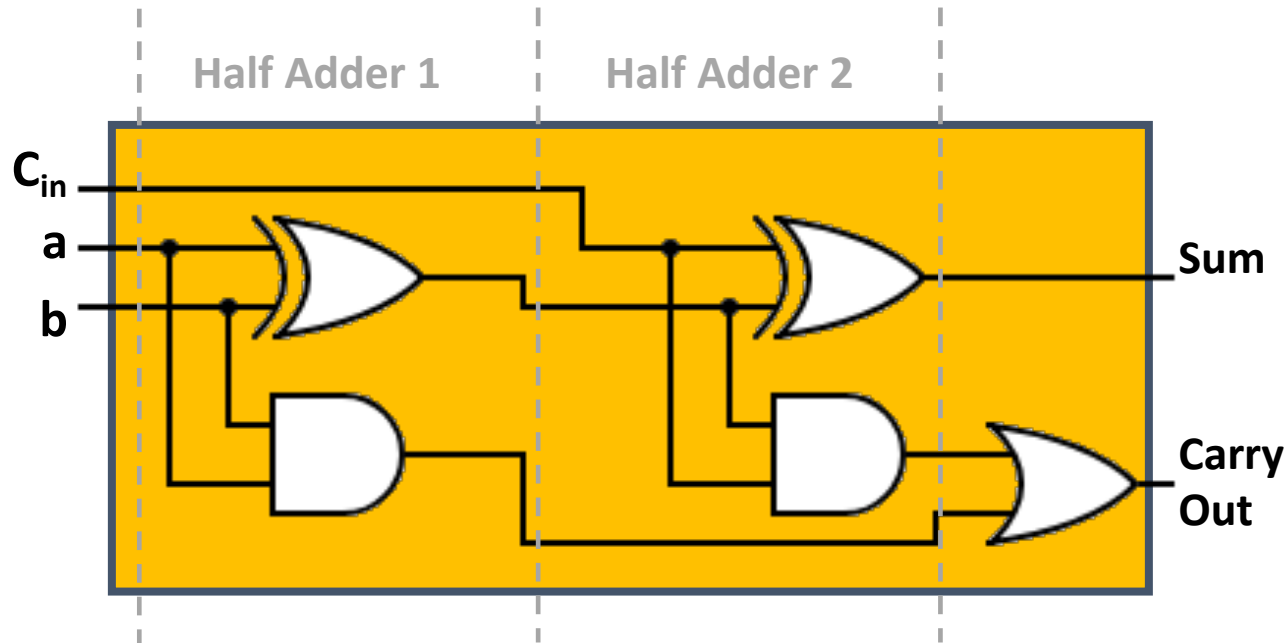
a	b	S_1	C_{out}^1	C_{in}	S_2	C_{out}^2	Carry Out (Expected)
0	0	0	0	0	0	0	0
0	1	1	0	0	1	0	0
1	0	1	0	0	1	0	0
1	1	0	1	0	0	0	1
0	0	0	0	1	1	0	0
0	1	1	0	1	0	1	1
1	0	1	0	1	0	1	1
1	1	0	1	1	1	0	1

Carry Out

a	b	s_1	C_{out}^1	C_{in}	s_2	C_{out}^2	C_{out}
0	0	0	0	0	0	0	0
0	1	1	0	0	1	0	0
1	0	1	0	0	1	0	0
1	1	0	1	0	0	0	1
0	0	0	0	1	1	0	0
0	1	1	0	1	0	1	1
1	0	1	0	1	0	1	1
1	1	0	1	1	1	0	1

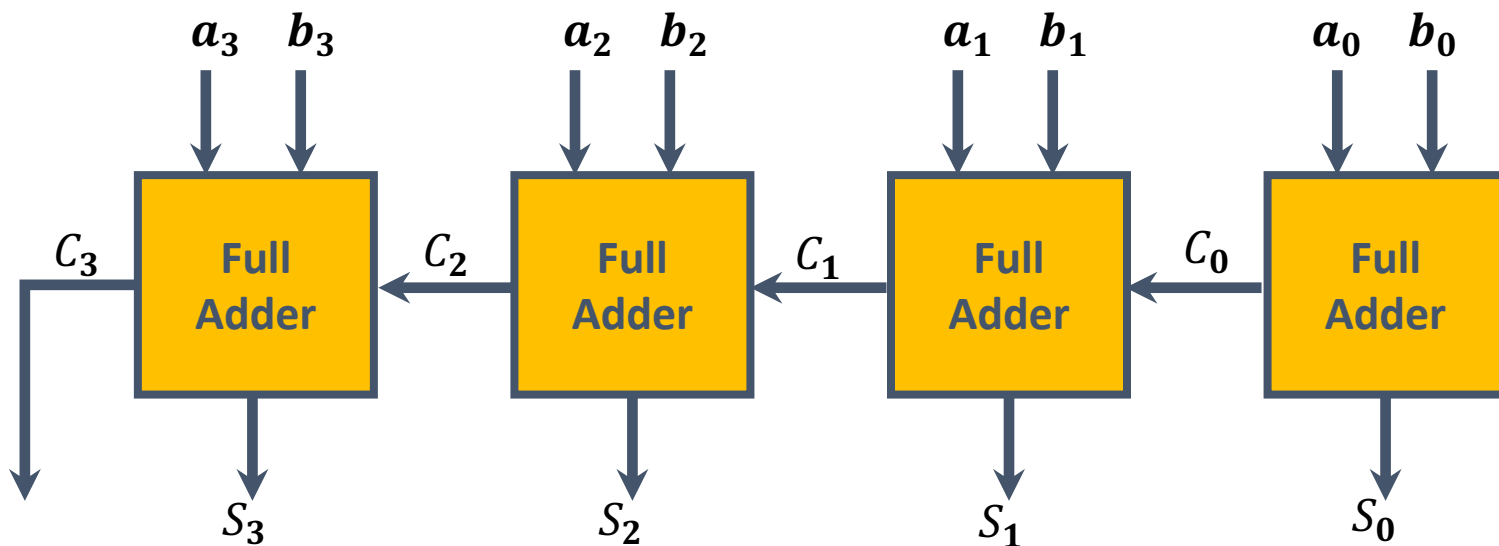
C_{out}^1	C_{out}^2	C_{out}
0	0	0
0	1	1
1	0	1
1	1	X

Full Adder with Half Adders



4-bit Adder

Number 1	a_3	a_2	a_1	a_0
Number 2	b_3	b_2	b_1	b_0



Ripple Carry Adder

Next...

Binary Subtractor

Computer Systems

Kasun Gunawardana

E-mail: kgg



University of Colombo School of Computing

Binary Subtractor

Subtractor

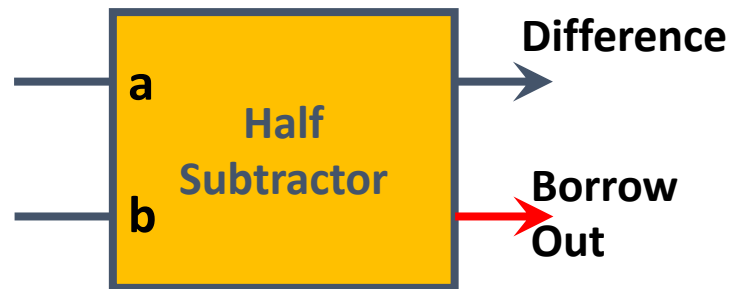
- Similar to Adders we can design,
 - Half Subtractor
 - Full Subtractor

Number 1	0	0	1	1	(Minuend)
Number 2	0	1	0	1	(Subtrahend)
Difference	0	1	1	0	
Borrow Out	0	1	0	0	

The borrow out is set when the subtractor needs to borrow from the next digit in a multi-digit subtraction.

Half Subtractor

- INPUTS: Two single bit numbers (two bits)
- OUTPUTS: Difference and Borrow out
- Does not consider *Borrowing in*.



Half Subtractor - Truth Table

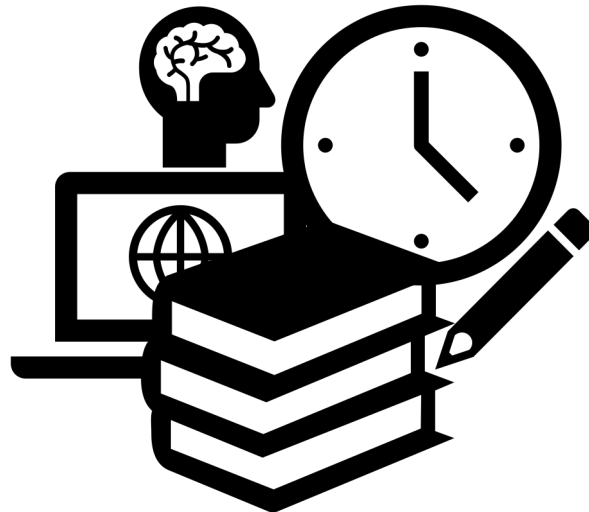
0	0	1	1
0	1	0	1
0	1	1	0
0	1	0	0

Inputs		Outputs	
a	b	Diff	B. Out
0	0	0	0
0	1	1	1
1	0	1	0
1	1	0	0

- The borrow out signal is set when the subtractor needs to borrow from the next digit in a multi-digit subtraction.

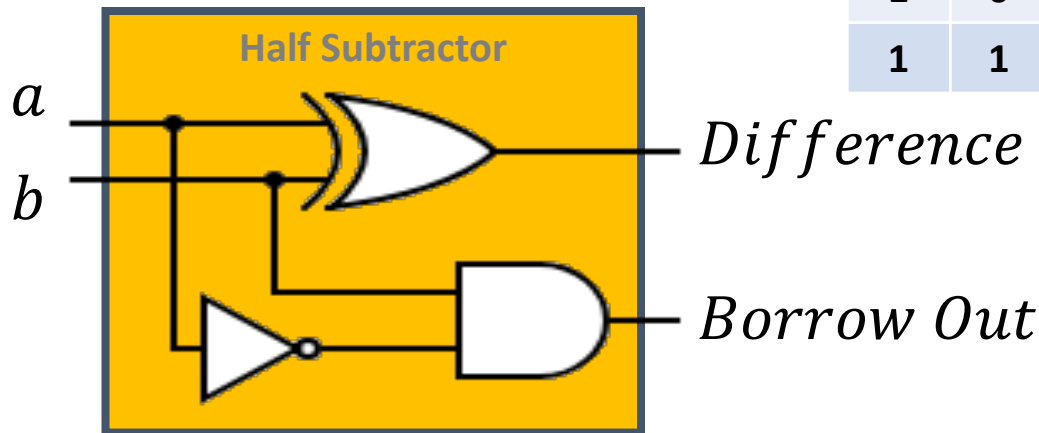
Half Subtractor - Circuit

- Try it out by yourself..
- Compare the truth table and the circuit with Half Adder
- Can Half Adder be employed to construct Half Subtractor ?



Half Subtractor

- Circuit Diagram

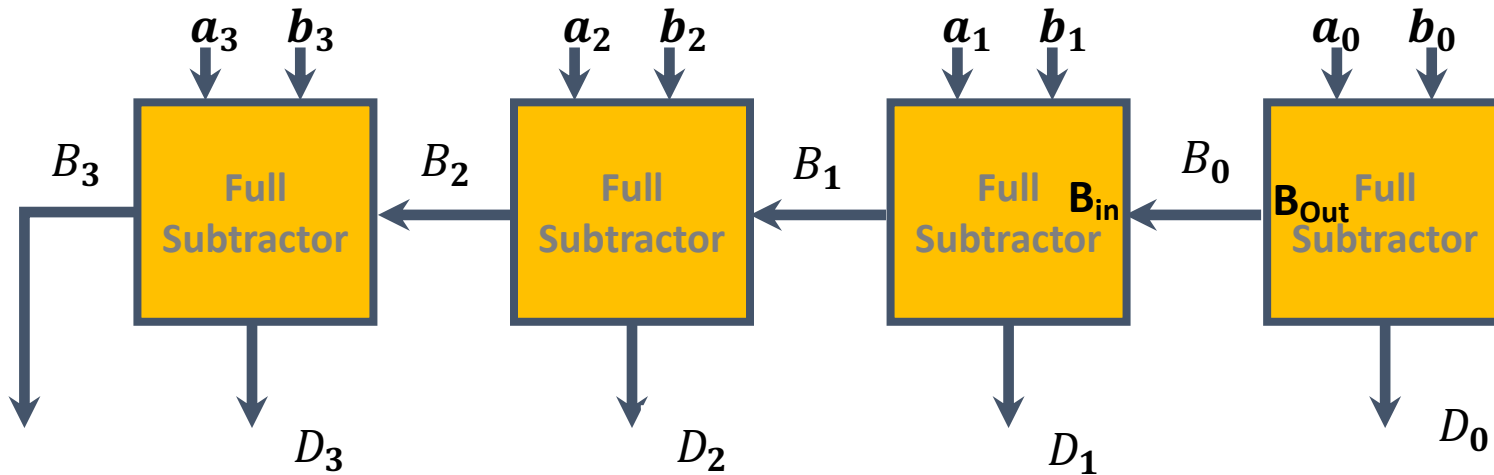


Inputs		Outputs	
a	b	Diff	B. Out
0	0	0	0
0	1	1	1
1	0	1	0
1	1	0	0

Note: Half subtractor circuit implements $a - b$ and not $b - a$

Full Subtractor

a_3	a_2	a_1	a_0
b_3	b_2	b_1	b_0
D_3	D_2	D_1	D_0



- Four bit subtractor – series of full subtractors should be able to subtract two binary numbers with multiple bits.

Full Subtractor

Inputs			Outputs	
a	b	B_{in}	$Diff$	B_{out}
0	0	0	0	0
0	0	1	1	1
0	1	0	1	1
0	1	1	0	1
1	0	0	1	0
1	0	1	0	0
1	1	0	0	0
1	1	1	1	1

B_{in} is set when the previous digit is borrowed from a

Full Subtractor

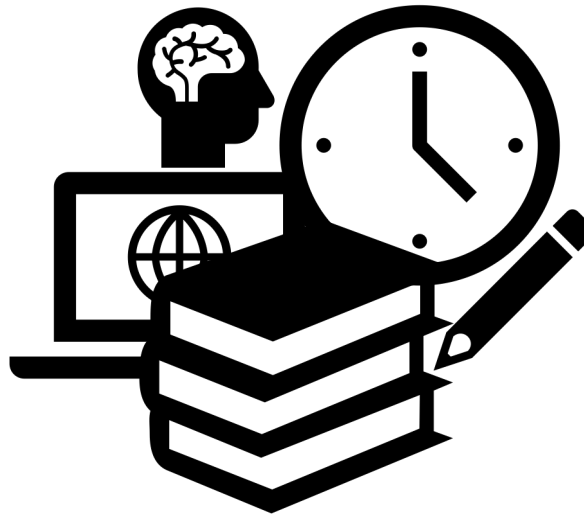
Inputs			Outputs	
a	b	B_{in}	$Diff$	B_{out}
0	0	0	0	0
0	0	1	1	1
0	1	0	1	1
0	1	1	0	1
1	0	0	1	0
1	0	1	0	0
1	1	0	0	0
1	1	1	1	1

Example

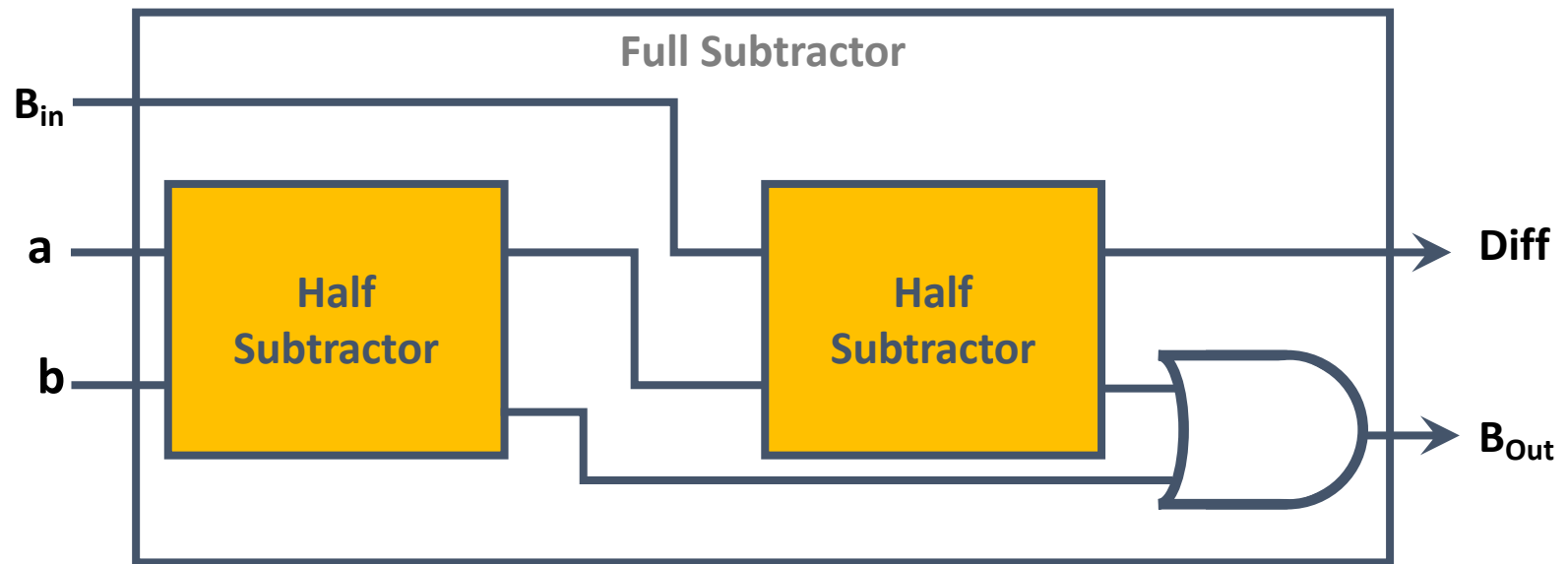
1	1	0	0
0	0	0	1
1	0	1	1

Full Subtractor – Alternative Appr.

- Construct Full Subtractor from Half Subtractors



Full Subtractor – Alternative Appr.



External reference: <https://en.wikipedia.org/wiki/Subtractor>

Multipliers

- How do you design a two bit multiplier?
- How do you design a multiplier for multiple bits?

ALU

- ALU contains many circuit units to perform different operations.
 - Arithmetic: $+$, $-$, $\times$, $\div$
 - Logic: $\&$, $|$, $\sim$, $\ll$, $\gg$

Next...

Decoders & Multiplexers

Computer Systems

Kasun Gunawardana

E-mail: kgg



University of Colombo School of Computing

Decoders & Multiplexers

Decoders

- Decoder is a special circuit unit to interpret a given code word.
- A binary code space of n bits can have 2^n number of codes.



- Decoder can interpret maximum of 2^n of codes by activating one of its 2^n number of output channels, given a n bit input code word.
- Each **minterm** activates a distinct output channel.

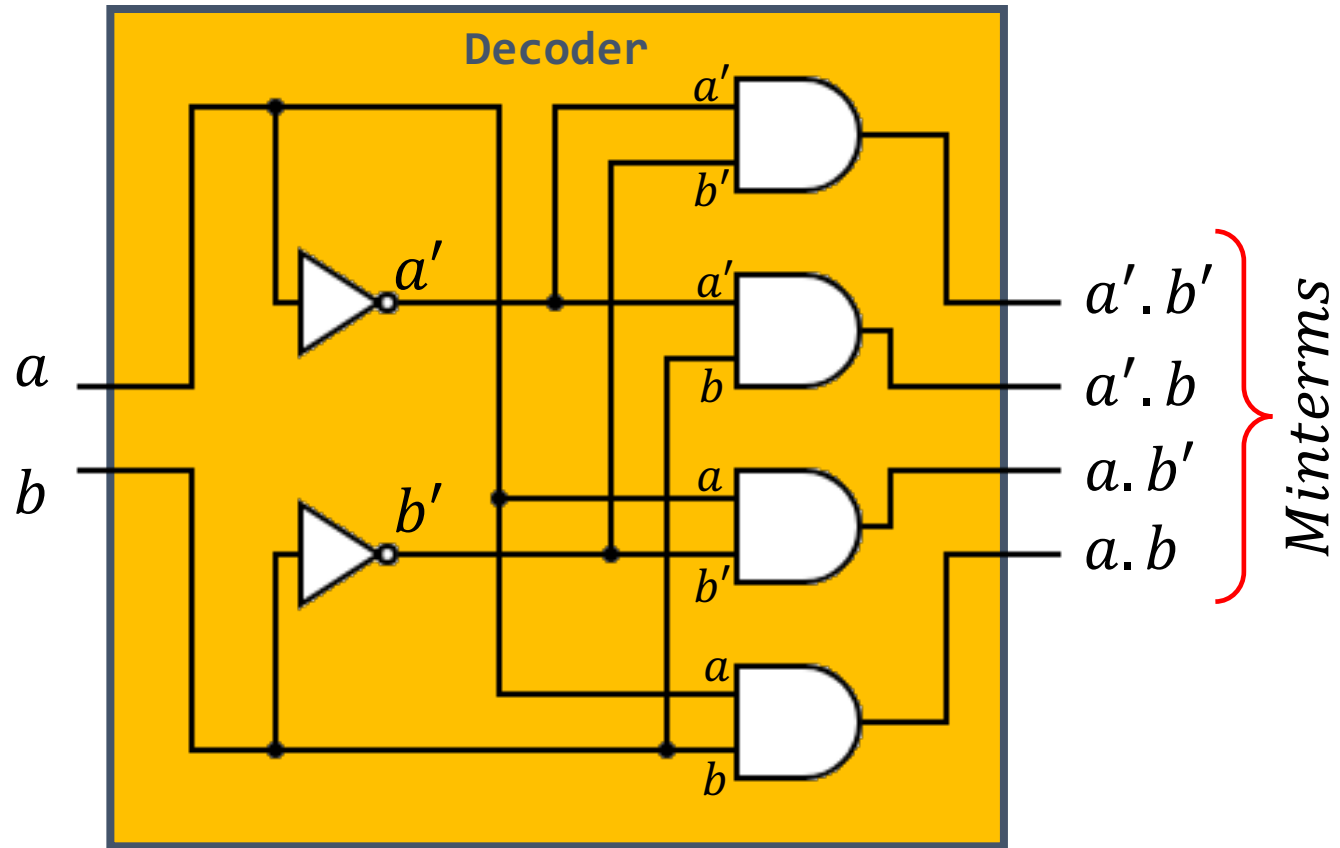
Decoders (Cont.)

- INPUTS: n bits
- OUTPUTS: 2^n bits

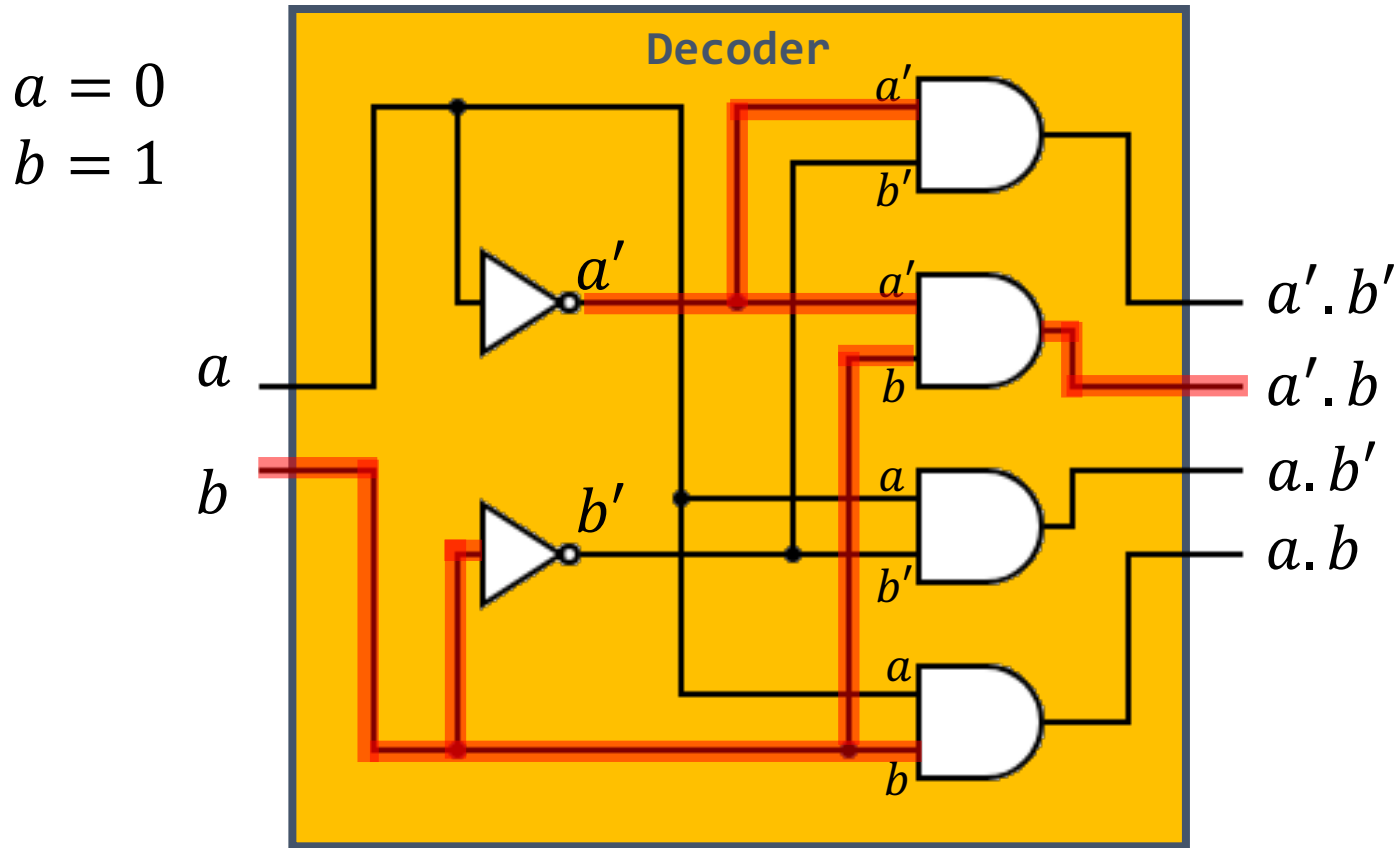


- Given an input code, only one output channel must be activated.
- Note: Decoders can have less than 2^n outputs.

2×4 Decoder



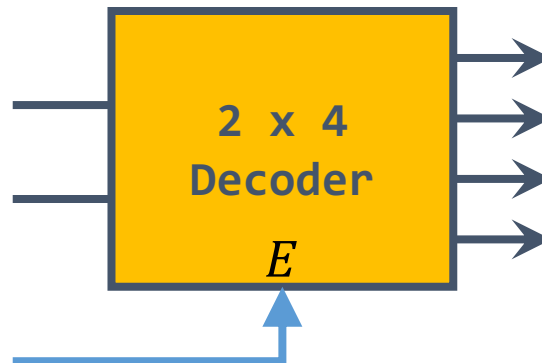
2x4 Decoder



Boolean state 1 channels are highlighted in RED

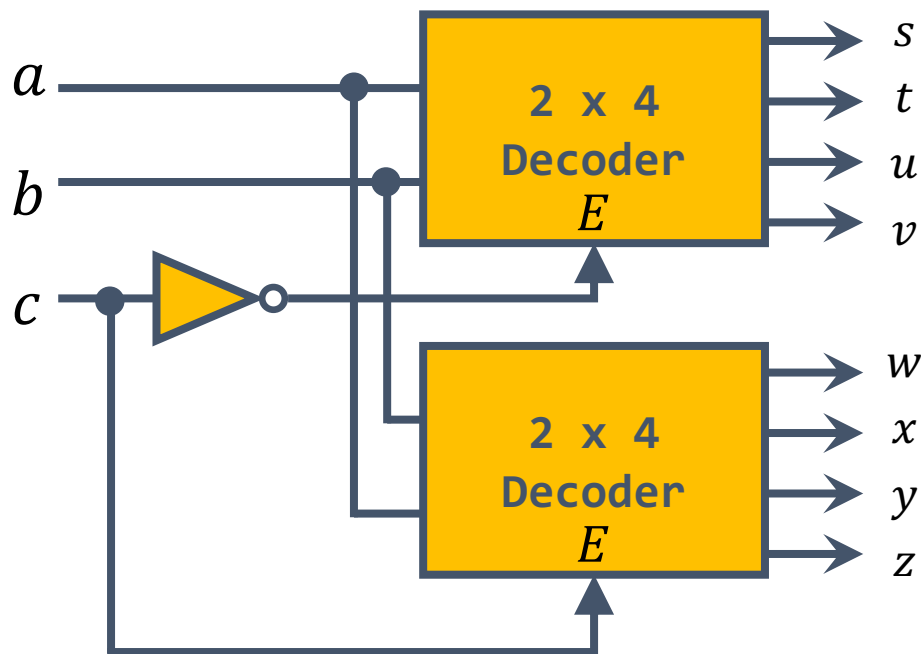
Decoder with Enable Input

- A single controller E input to enable/ disable
 - If $E = 1$ then the decoder will operate normal
 - If $E = 0$ then the decoder output will be inactive



Exercise..

- Derive the truth table for the following circuit and discuss the equivalent single circuit unit that you can use.



Answer

- It's your turn...

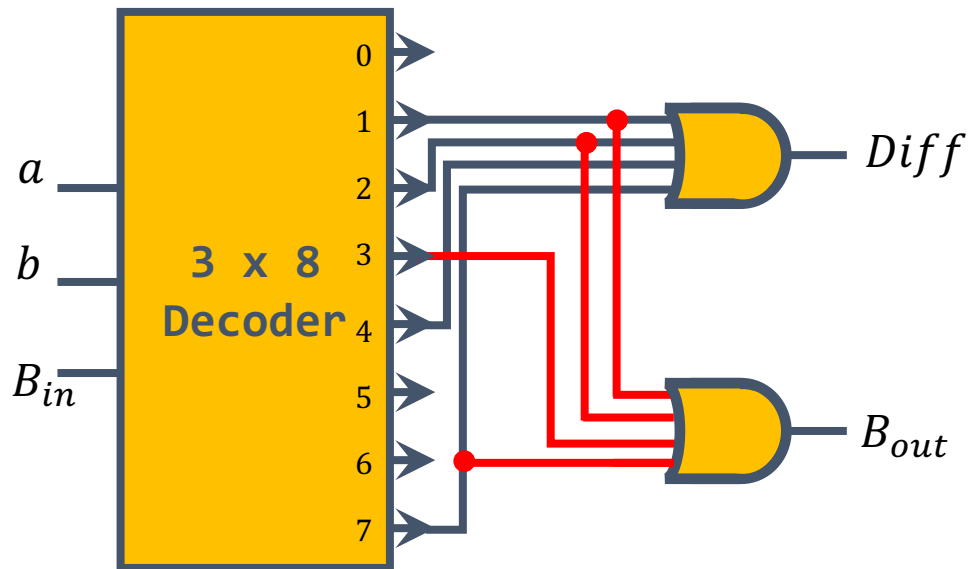
Decoder as Circuit Builder

- Any Boolean function can be represented in Sum of Minterms form.
- A decoder with n inputs can generate all the Minterms for n input variables.
- Logical Sum can be provided with an external **OR** gate.

Full Subtractor with Decoder

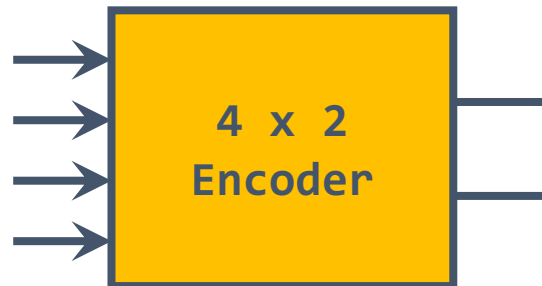
Inputs			Outputs	
a	b	B _{in}	Diff	B _{out}
0	0	0	0	0
0	0	1	1	1
0	1	0	1	1
0	1	1	0	1
1	0	0	1	0
1	0	1	0	0
1	1	0	0	0
1	1	1	1	1

$$Diff = \sum (1, 2, 4, 7) \quad B_{out} = \sum (1, 2, 3, 7)$$



Encoders

- Encoder performs the inverse operation of a decoder.
- Encoder can have 2^n of inputs and maximum of n output channels.

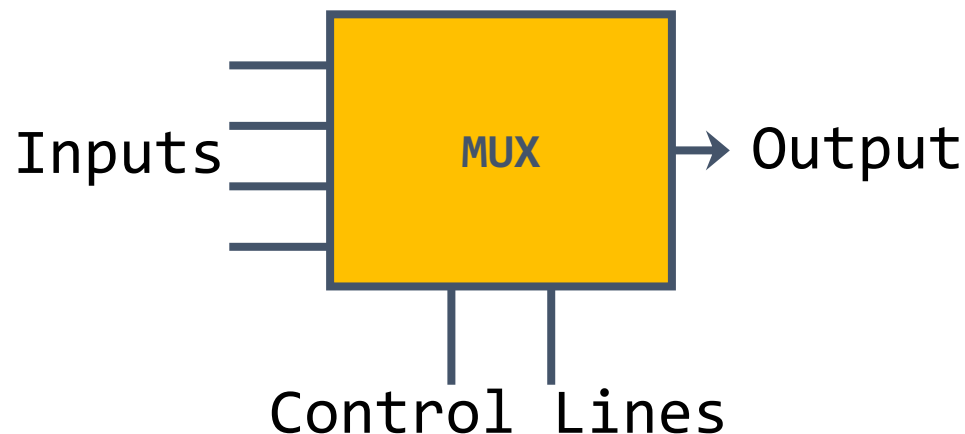


Exercise..

- Design a circuit for 8 x 3 Encoder
 - Search..!
 - Read..!
 - Explore..!

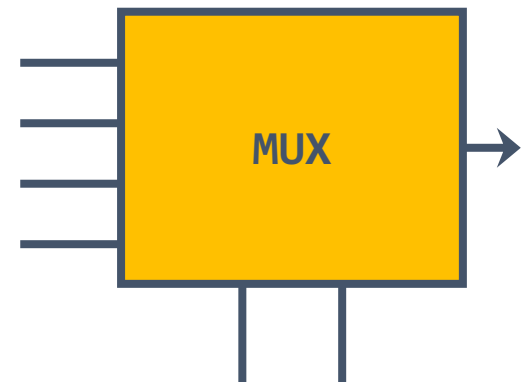
Multiplexers

- One of many input channels is directed to the single output channel.
 - Several input channels
 - Single output channel
 - Several control lines to direct an input channel to the output channel



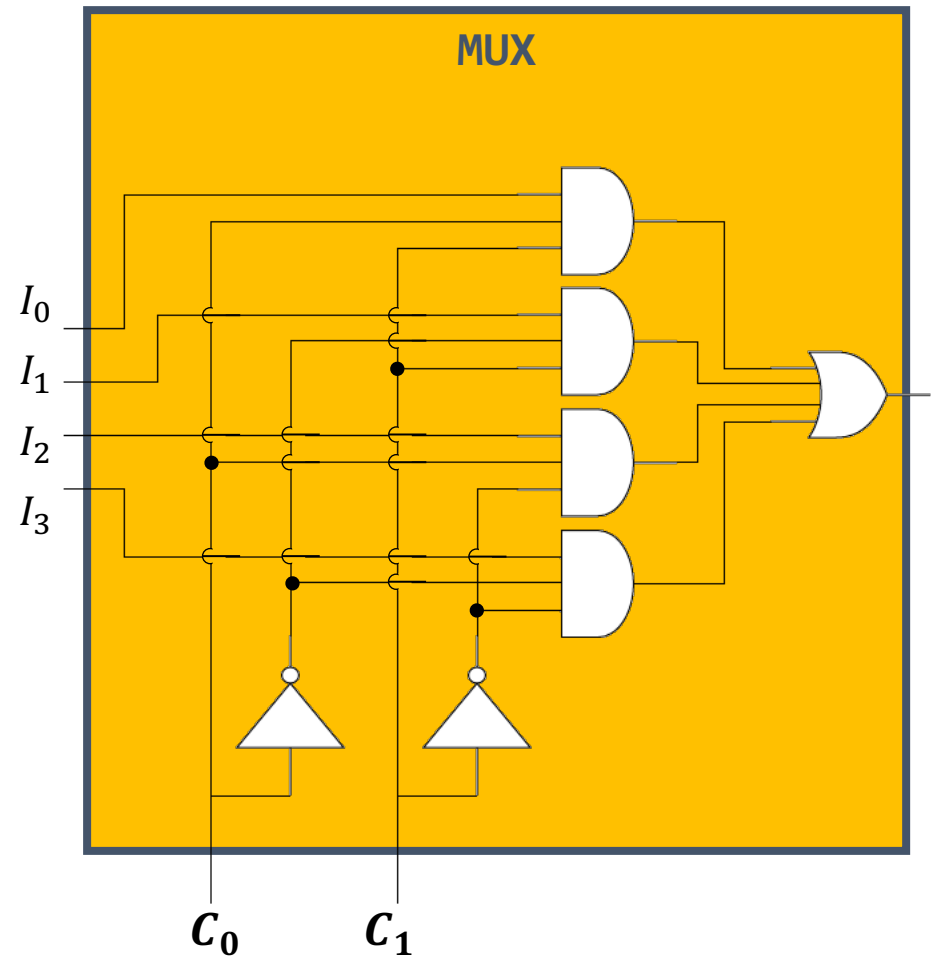
Multiplexer

- Depends on the value set on control lines, only one input will be directed to the output.
- Dedicated control line pattern as the pass code for each input.
- 4 patterns from 2 control lines.
- 4 inputs can be directed using 2 control lines.
- Thus, n control lines for 2^n inputs.



Multiplexer Circuit

Input	Control Code		Input Activation
	C_0	C_1	
I_0	0	0	$I_0 \cdot C_0' \cdot C_1'$
I_1	0	1	$I_1 \cdot C_0' \cdot C_1$
I_2	1	0	$I_2 \cdot C_0 \cdot C_1'$
I_3	1	1	$I_3 \cdot C_0 \cdot C_1$



Reading Exercise

- Read about Demultiplexers.

Exercise..

- What are the uses of the following circuit units in a computer system?
 - Decoder
 - Multiplexer

Thank You..!

Computer Systems

Kasun Gunawardana

E-mail: kgg

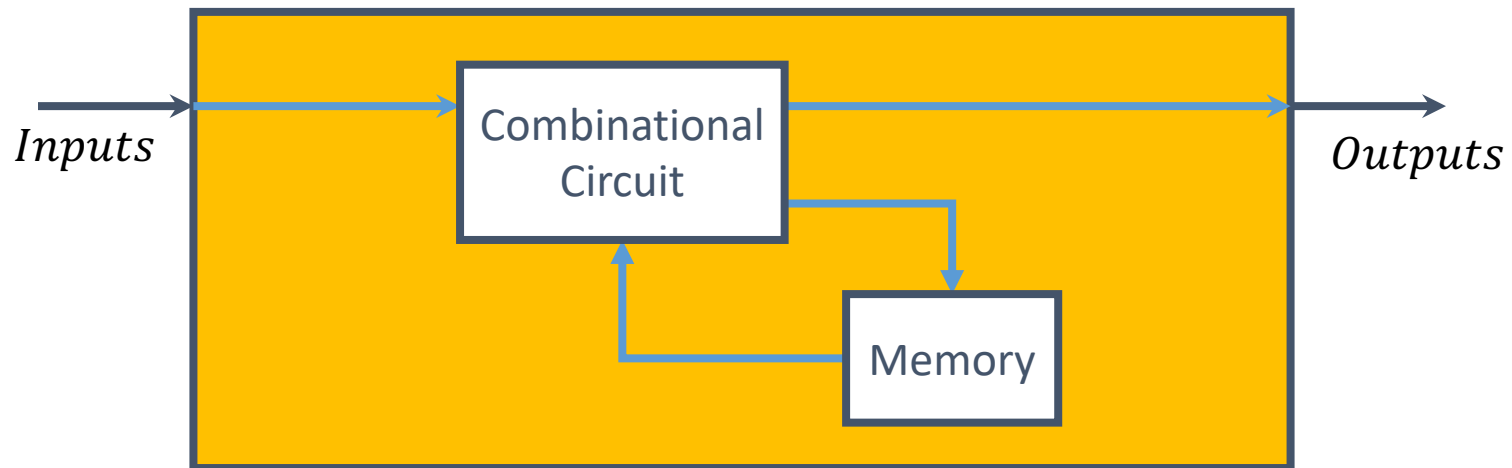


University of Colombo School of Computing

Sequential Logic Circuits

Combinational / Sequential

- **Combinational Circuits:** Outputs only depend on immediate inputs.
- **Sequential Circuits:** Outputs depend not only on immediate inputs, but also previous outputs.



Sequence

- Memory element's transition among states.
- The next state of the memory element is determined by its inputs and the present state.
- A circuit can be seen as a time sequence of inputs, outputs, and internal states.

Sequential Circuits

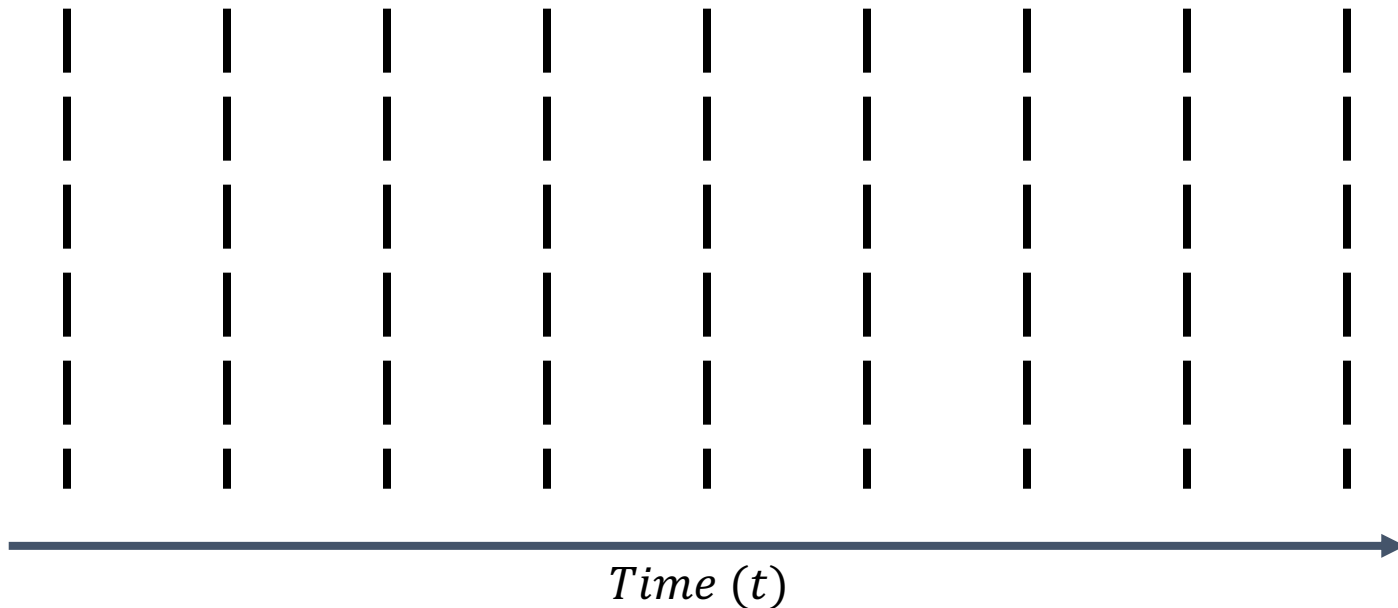
- Asynchronous – State changes immediately after inputs are changed.
- Synchronous – State changes only within discrete time intervals.

Asynchronous Sequential Circuits

- The state of a circuit changes as soon as the inputs change.
- Faster due to immediate state changes.
 - Only propagation delays affects the speed.
- A combinational circuit with feedback.
- Due to this nature, circuit may become unstable.
- Difficult to design circuits with Asynchronous Sequential circuits.

Synchronous Sequential Circuits

- Synchronization is achieved by making the circuit changes its state at discrete intervals of time.

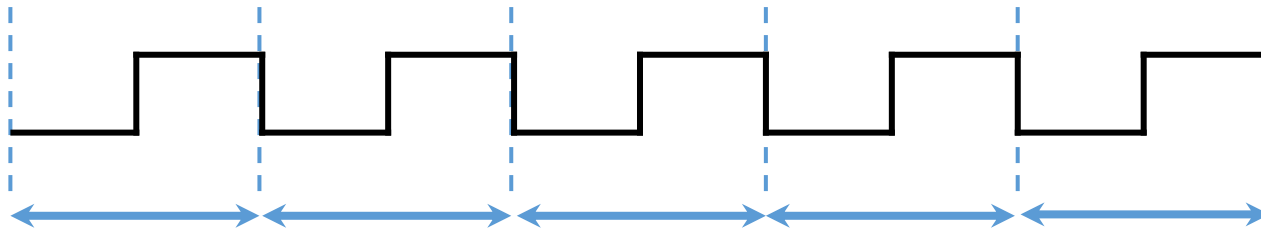


Clocked Synchronous Seq. Circuits

- Separate controller signal for the synchronization.
- A clock can be used to generate signals (pulses).
- Storage elements' states are changed with clock pulses.
- Clock signal is connected to all the storage elements in the circuit.
- Pulses determine **when** the state changes occur.
- Other inputs determine **what** changes will be made.

Clock Signal

Clock Signal



Discrete Time Intervals

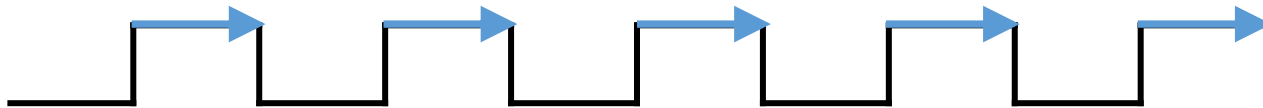
Flip - Flop

- A storage elements used in Clocked Synchronus Sequential Circuits (C.S.S.C) are called **Flip-Flops**.
- It can store one bit of information.
- A C.S.S.C may contain many flip-flops.
- Propagation delays and Clock interval are critical.
- State changes are triggered by the clock pulse transition (Edge sensitive).



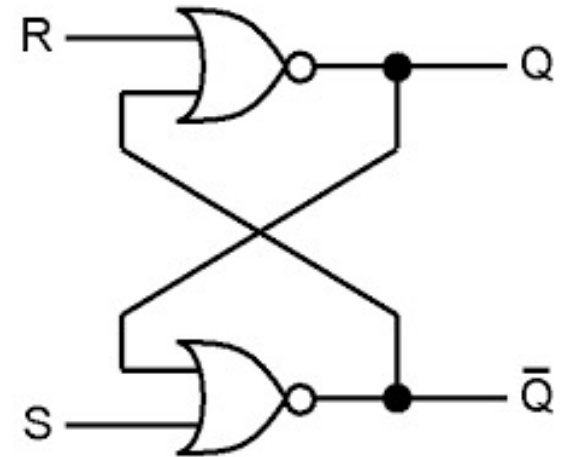
Latch

- A storage element that operates on signal levels rather than signal transition (level sensitive), is called a latch.
- Not operates on clock edges.
- Asynchronus device (Not synchronus).
- Latches are the building blocks for Flip-Flops.



S-R Latch

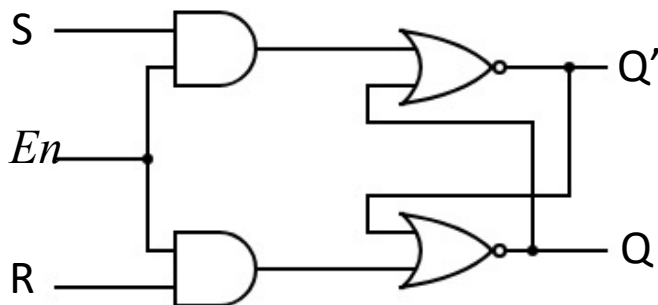
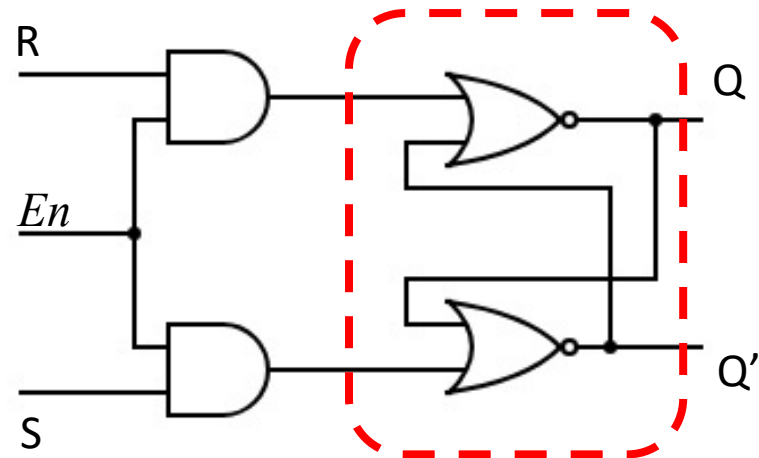
- **S** – Set, **R** – Reset
- Two cross coupled NAND gates can also be used.
- Two outputs; One is the complement of the other.
- When $Q=1 \rightarrow$ Set State
- Otherwise Reset State



S	R	$Q(t+1)$
0	0	$Q(t)$
0	1	0
1	0	1
1	1	Forbidden

S-R Latch with Control Signal

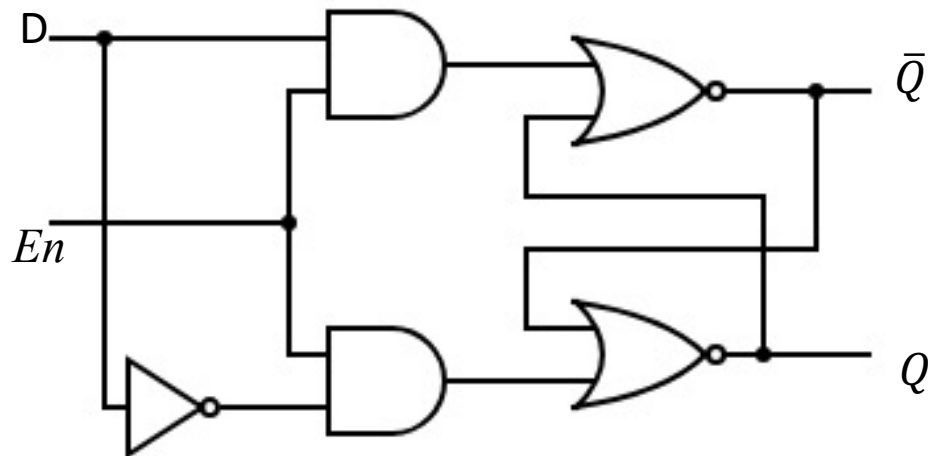
- En – Enable
- When $En = 0$ circuit is a SR Latch with $S=0$ and $R=0$
- En should be triggered to make the S-R Latch operational.



En	S	R	Q(t+1)
0	X	X	Q(t)
1	0	0	Q(t)
1	0	1	0
1	1	0	1
1	1	1	Forbidden

D Latch

- Eliminate undesirable condition of S-R Latch



En	D	Q(t+1)
0	X	Q(t)
1	0	0
1	1	1

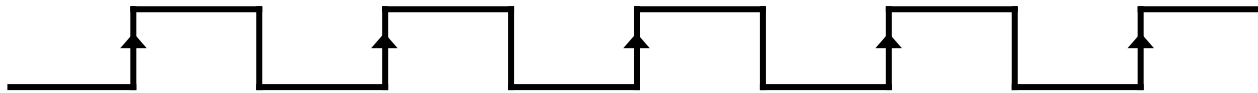
- Value in goes to out (Q) when the circuit is enabled.
- When disabled value Q remains unchanged.

Latches with Clock

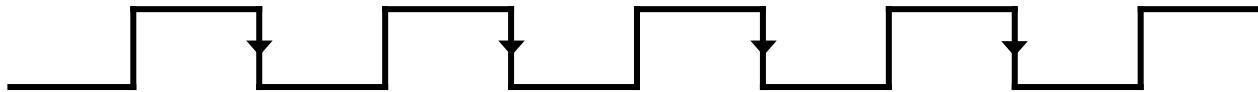
- A clock can be used as a control signal in latches.
- When control signal is active latches make transitions.
- If inputs are changed during clock pulse active period, then within a single clock pulse the latch generates two outputs.
- Latches have the problem of level sensitivity.
- Flip-Flops solves this problem by being edge sensitive.

Edge Sensitive

- Positive Edge (Rising Edge)

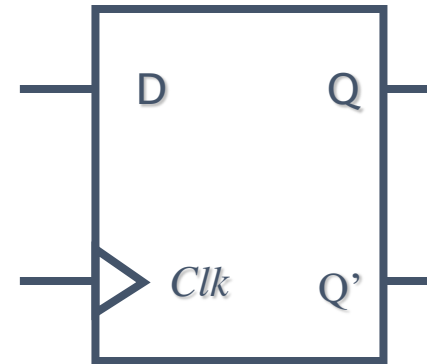


- Negative Edge (Falling Edge)



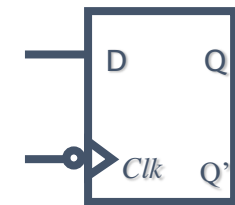
D Flip-Flop

- Edge triggered storage device.
- Store only one bit.



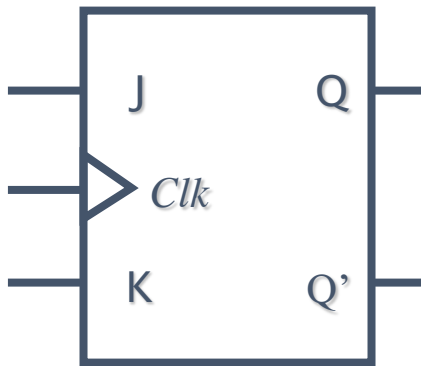
D	Clk	Q(t+1)
0	0	Q(t)
0	1	0
1	0	Q(t)
1	1	1

D	Q(t+1)
0	0
1	1



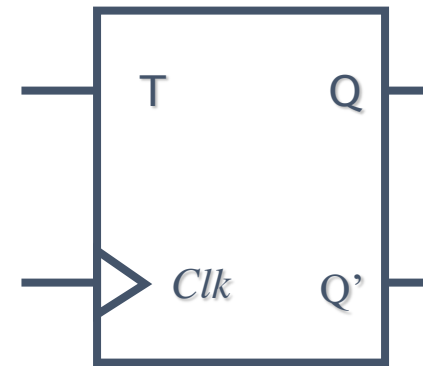
Negative Edge Sensitive
D Flip-Flop

J-K Flip-Flop and T Flip-Flop



J-K Flip-Flop

J	K	$Q(t+1)$
0	0	$Q(t)$
0	1	0
1	0	1
1	1	$Q'(t)$



T Flip-Flop

T	$Q(t+1)$
0	$Q(t)$
1	$Q'(t)$

Next...

Common Applications of Flip-Flops

Computer Systems

Kasun Gunawardana

E-mail: kgg



University of Colombo School of Computing

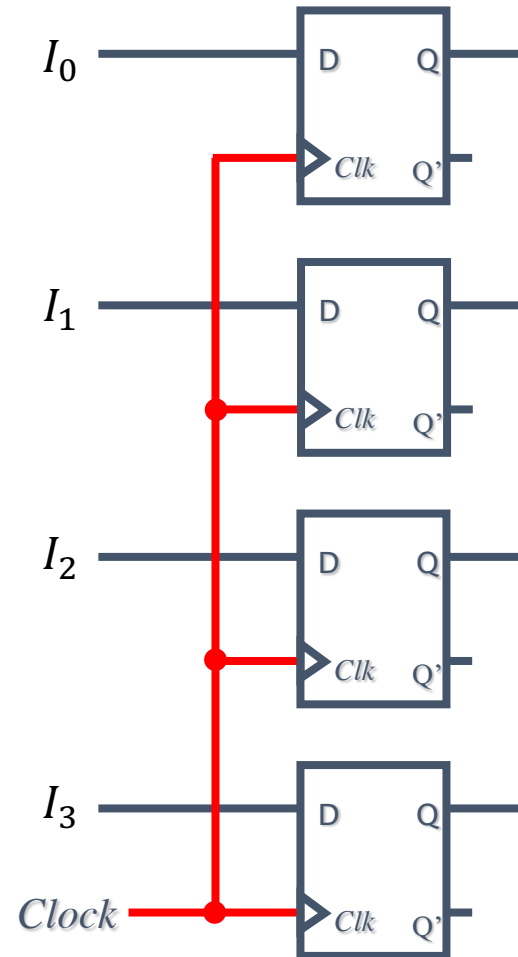
Common Applications of Flip-Flops

Registers

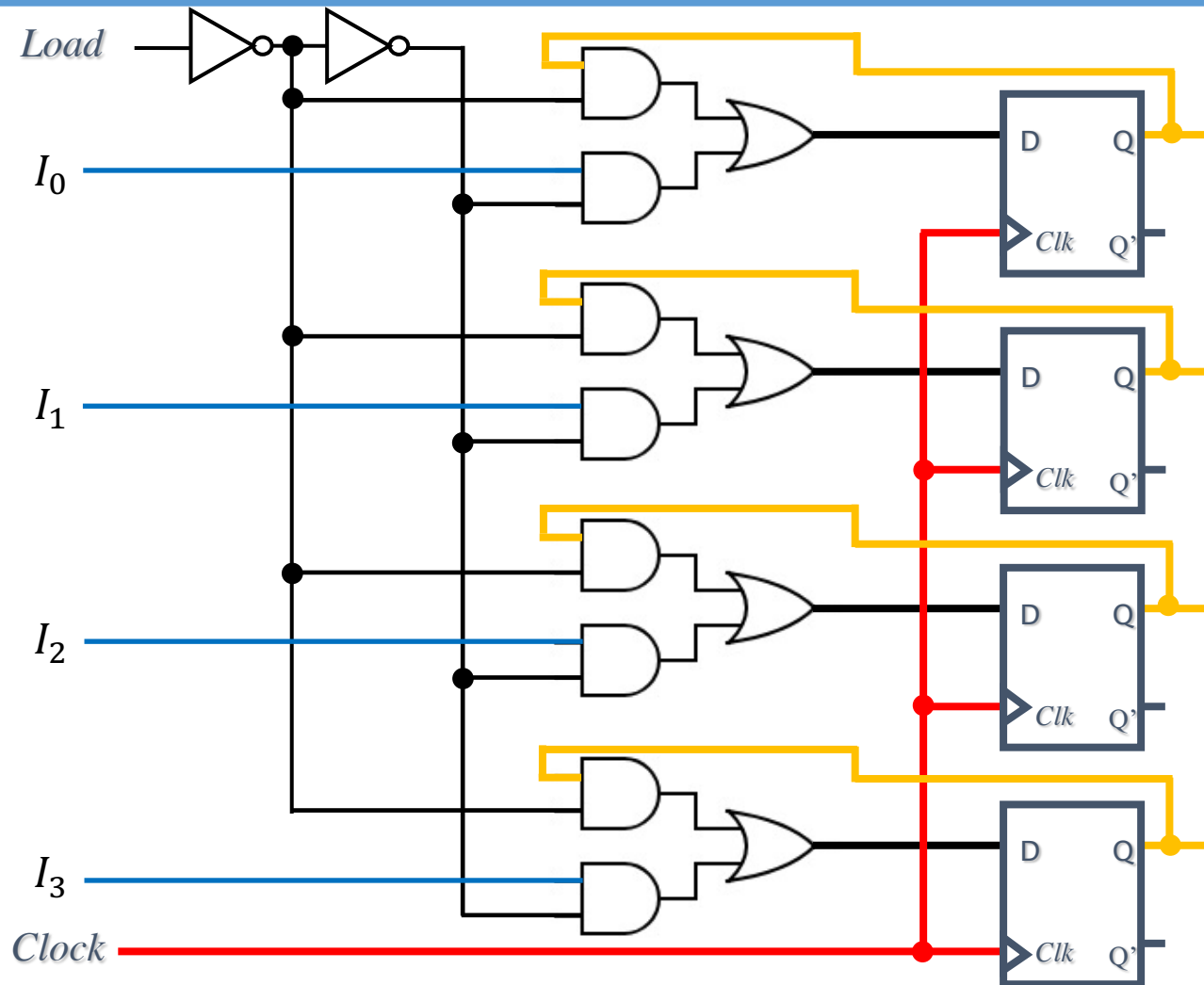
- A series of flip-flops with a common clock input is considered as a Register.
 - Ex. A 32 bit register consists of 32 flip-flops.
- A combinational circuit may combine with a register to aid additional operations.
 - Flip-flops to hold the information
 - Combinational circuit to process/ transfer the information

Storage Register

- Simplest Register
- A group of D Flip-Flops
- Binary inputs will be stored at the flip-flops with a clock signal.
- Stored information can be read at anytime.
- Can have separate asynchronus input signal to clear all flip-flops.



4 bit Register with Parallel Load



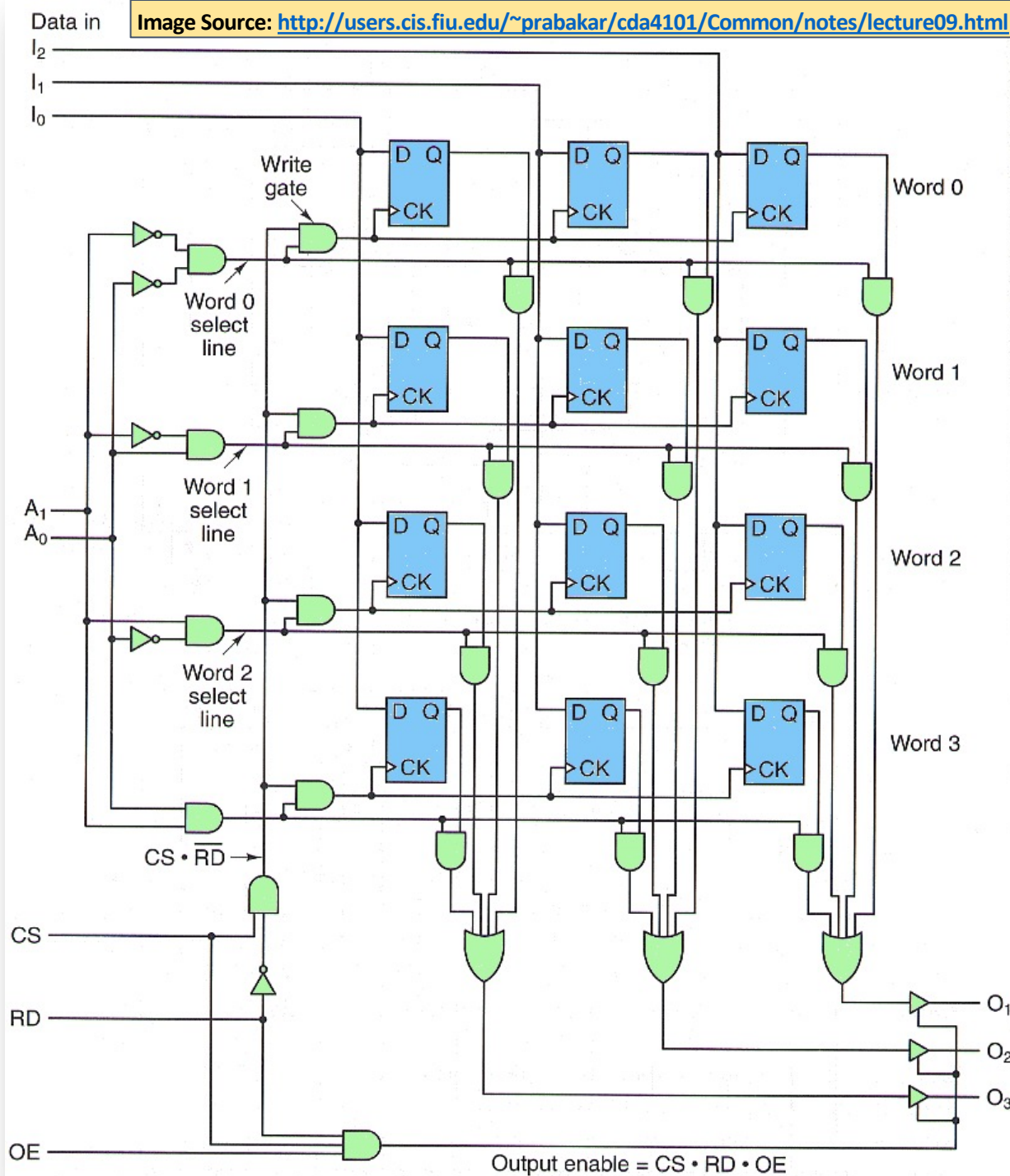
Shift Registers

Explore about “*Shift Registers*”

Start from

- https://en.wikipedia.org/wiki/Shift_register

RAM



4 words
Word Size = 3 bits

Data bus: $I_2 - I_0$

Address bus = $A_1 - A_0$

CS: Internal Activation
RD: Read Enable
OE: Output Enable

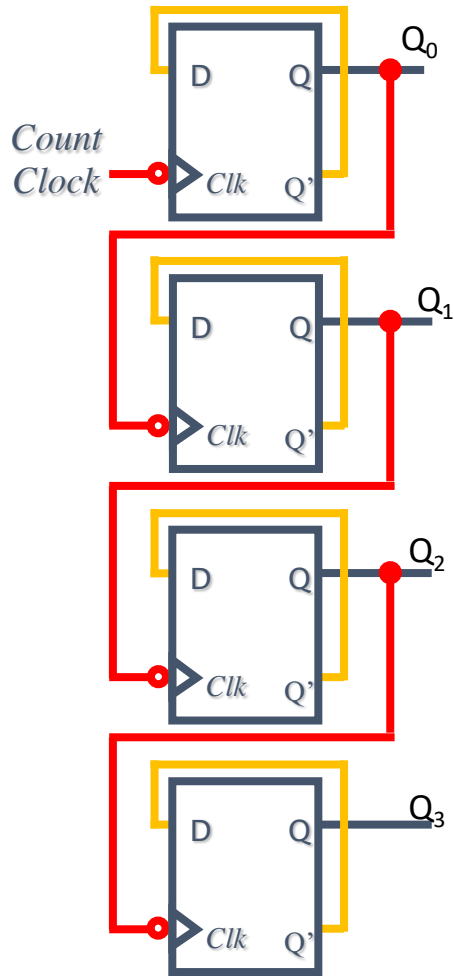
Recognize the Decoder,
Multiplexers here...

Counters

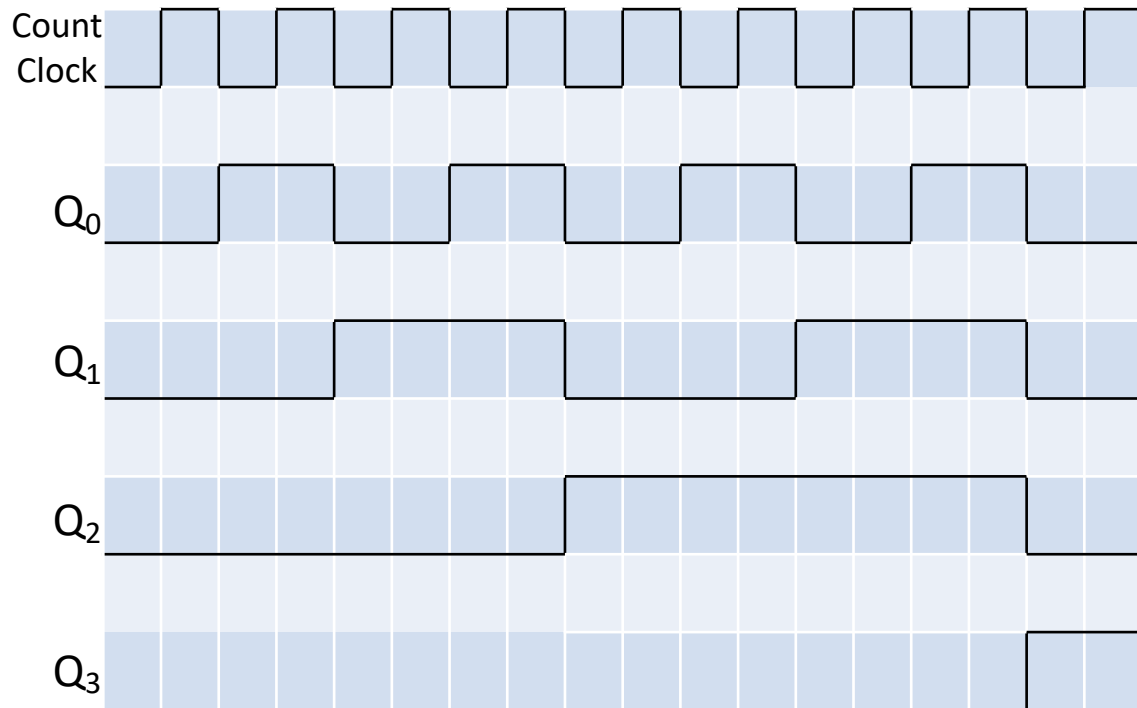
An ordered set of Flip-Flops can produce a defined sequence of states upon a series of triggers.

- **Ripple Counters:** Each Flip-Flop is triggered not by a clock, but by the another flip-flop output transition.
- **Synchronous Counters:** Flip-Flops are triggered by a common clock.

Binary Ripple Counter

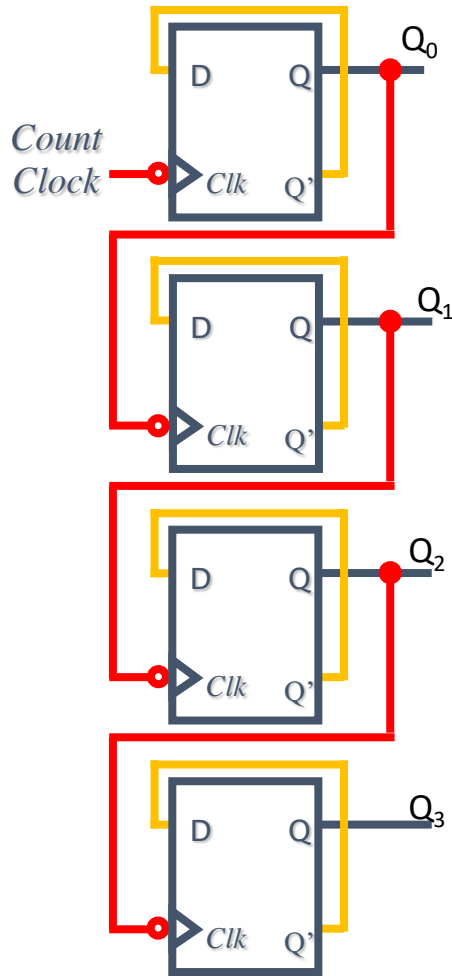


Timing Diagram

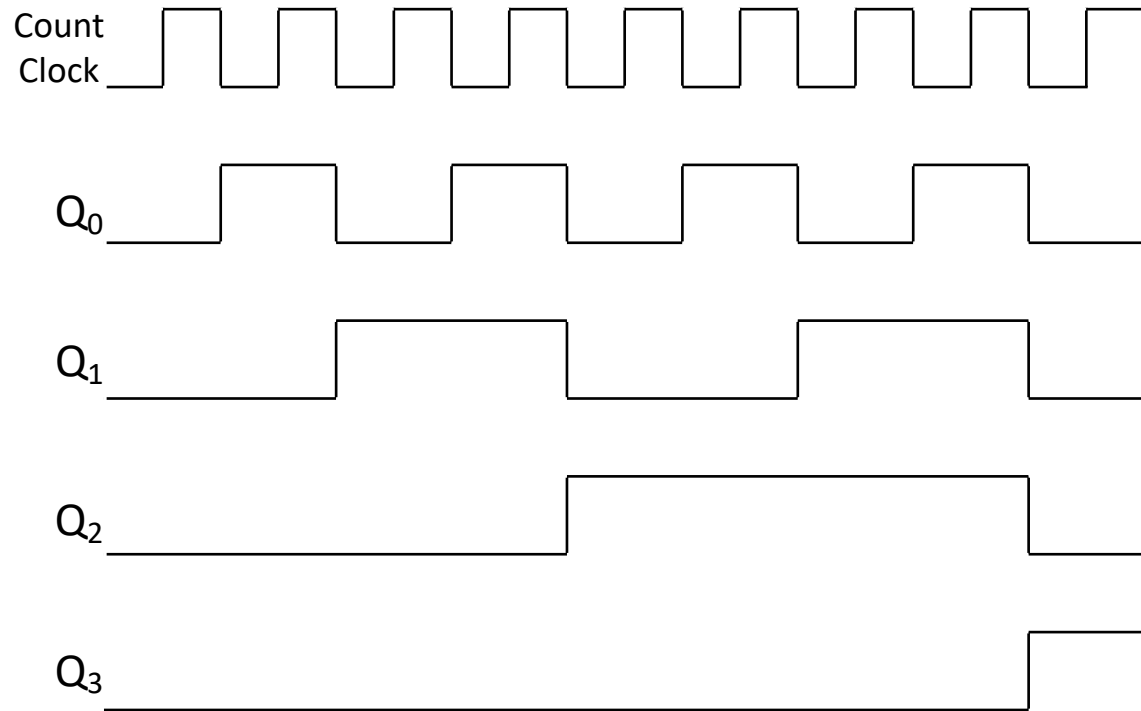


Note: Negative edge sensitive D Flip-Flops

Binary Ripple Counter (Cont.)

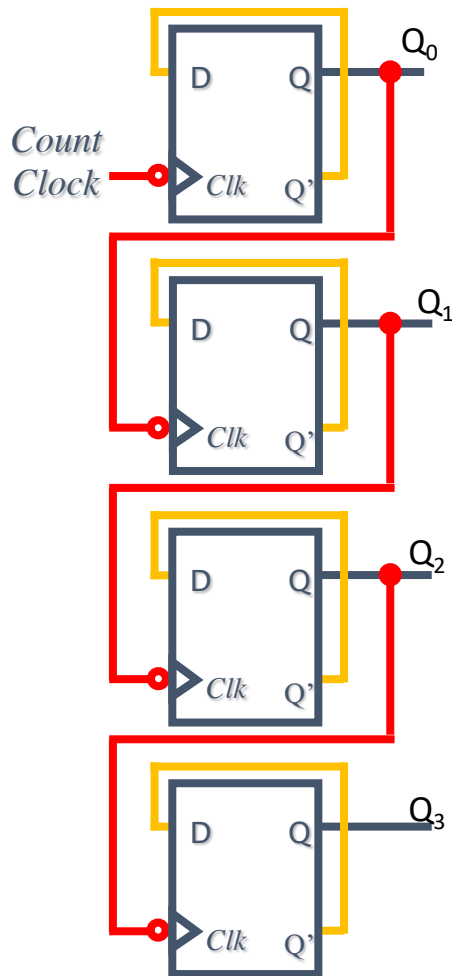


Timing Diagram

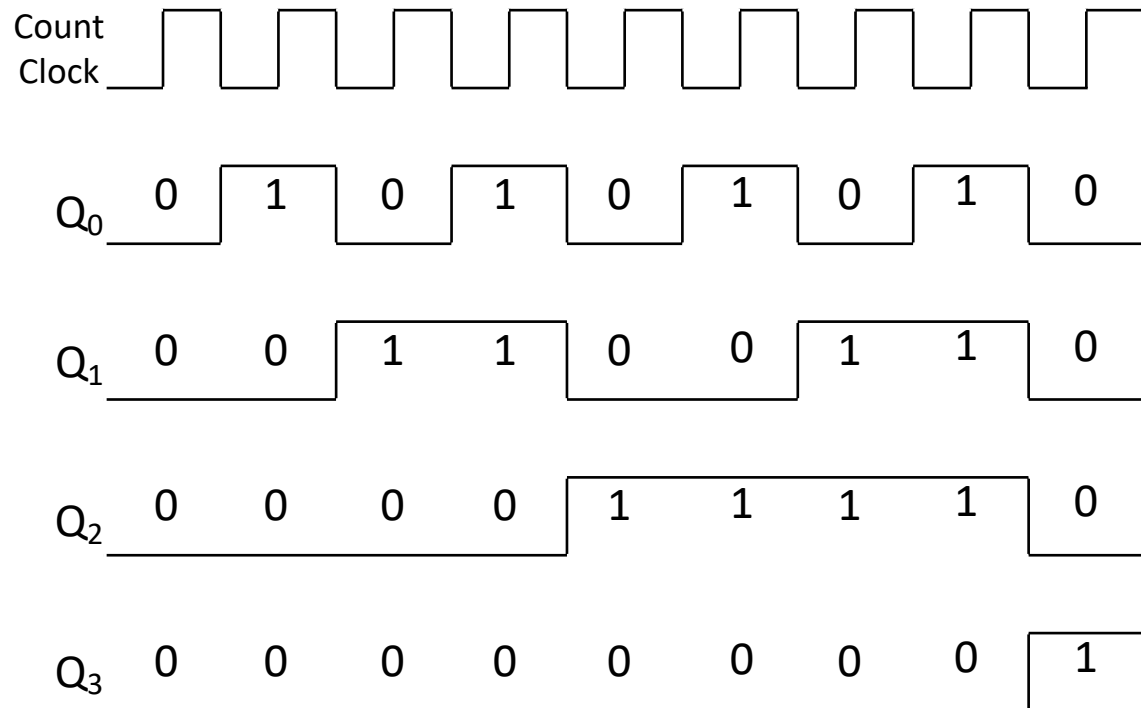


Note: Negative edge sensitive D Flip-Flops

Binary Ripple Counter (Cont.)

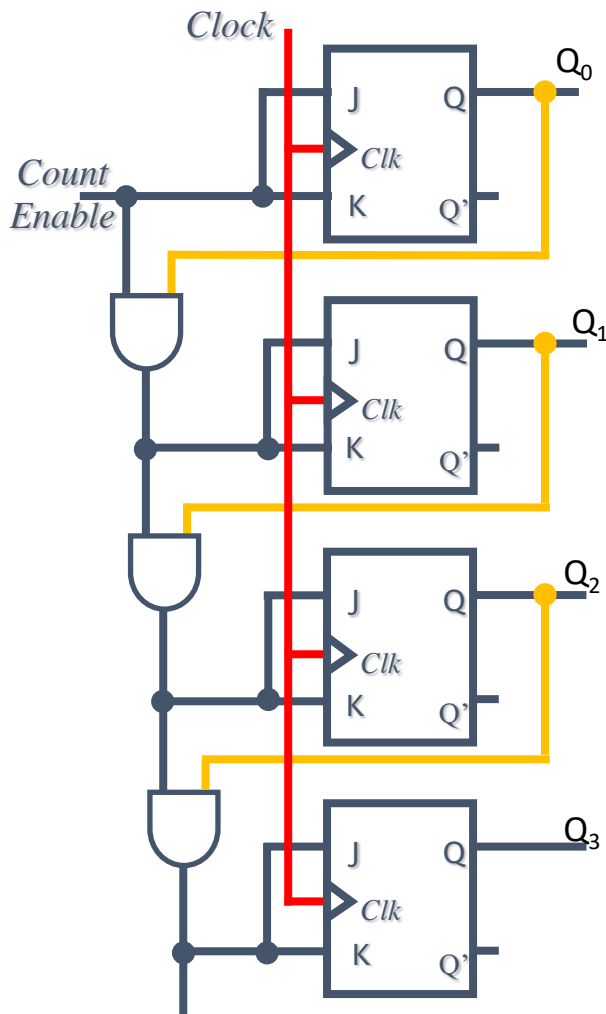


Timing Diagram



Note: Negative edge sensitive D Flip-Flops

Synchronous Binary Counter



Set Up

- Q_0, Q_1, Q_2, Q_3 are zeros at the beginning
- Positive Edge triggered J-K Flip-Flops
- *Count Enable* is set 1
- Common Clock is connected to all Flip-Flops

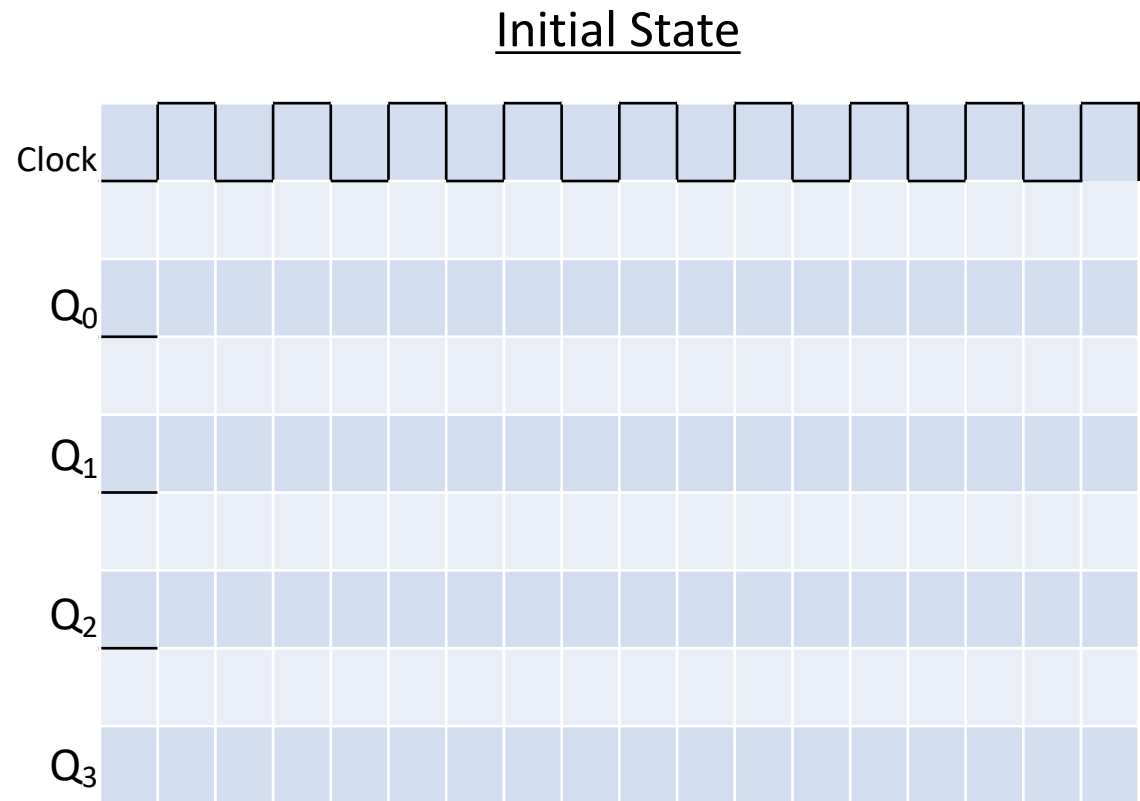
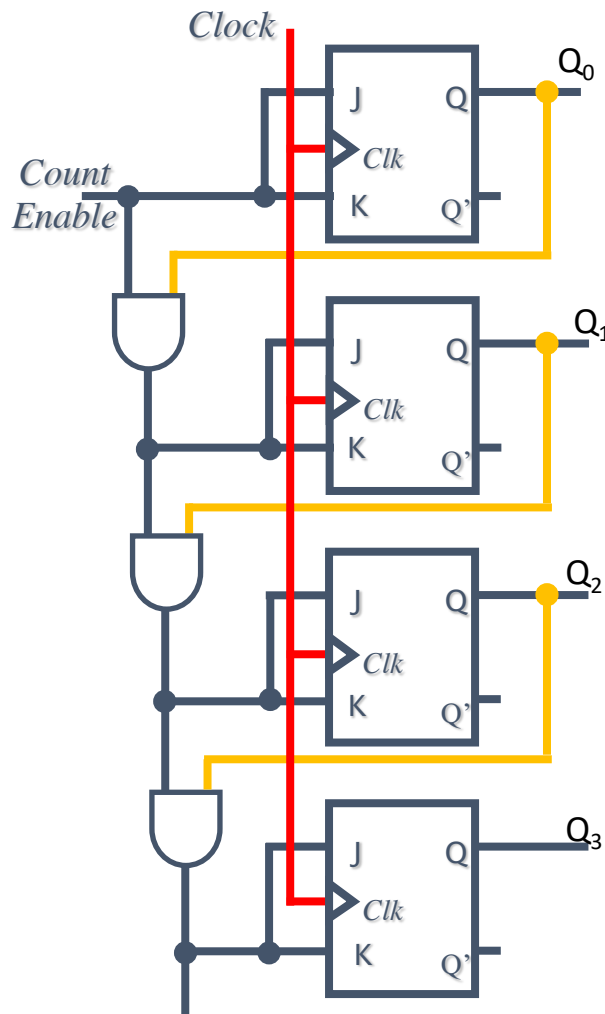
At the very first positive edge of the clock pulse

- First Flip-Flop's J and K inputs are at logical 1
- Second Flip-Flop's J and K inputs are at logical 0
- Third Flip-Flop's J and K inputs are at logical 0
- Fourth Flip-Flop's J and K inputs are at logical 0

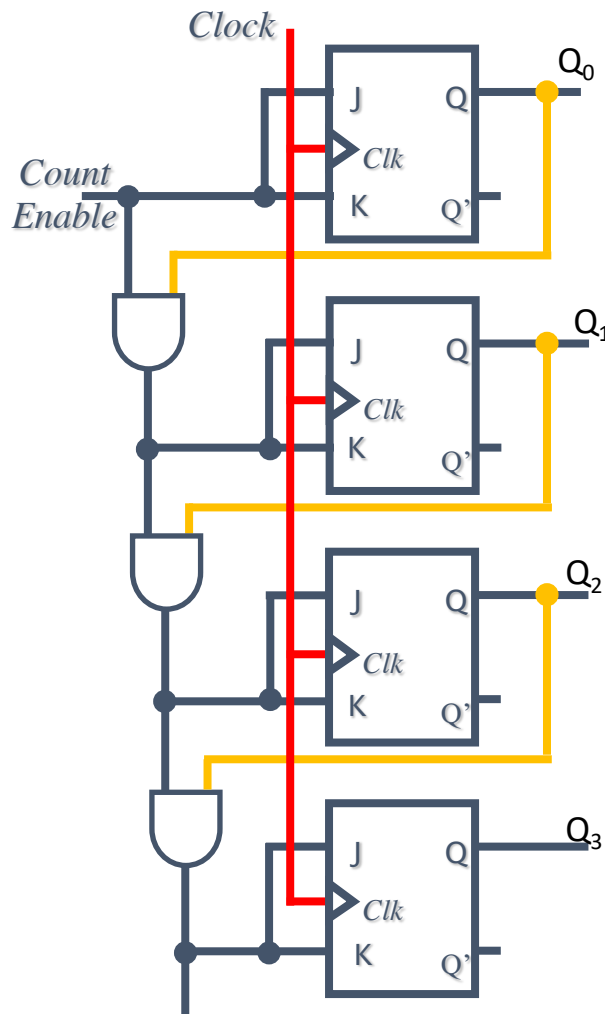
Effect

- Only the first Flip-Flop changes its value
- All the other Flip-Flops remain unchanged
- States are unchanged until it sees the next positive edge clock signal

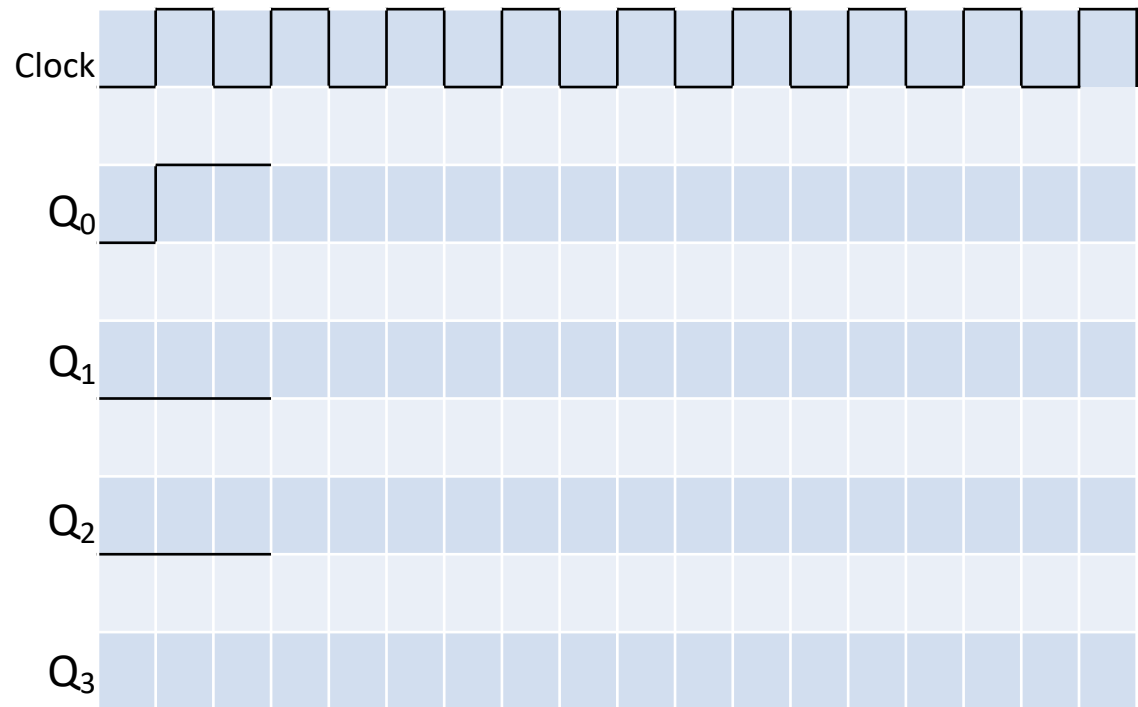
Synchronous Binary Counter (Cont.)



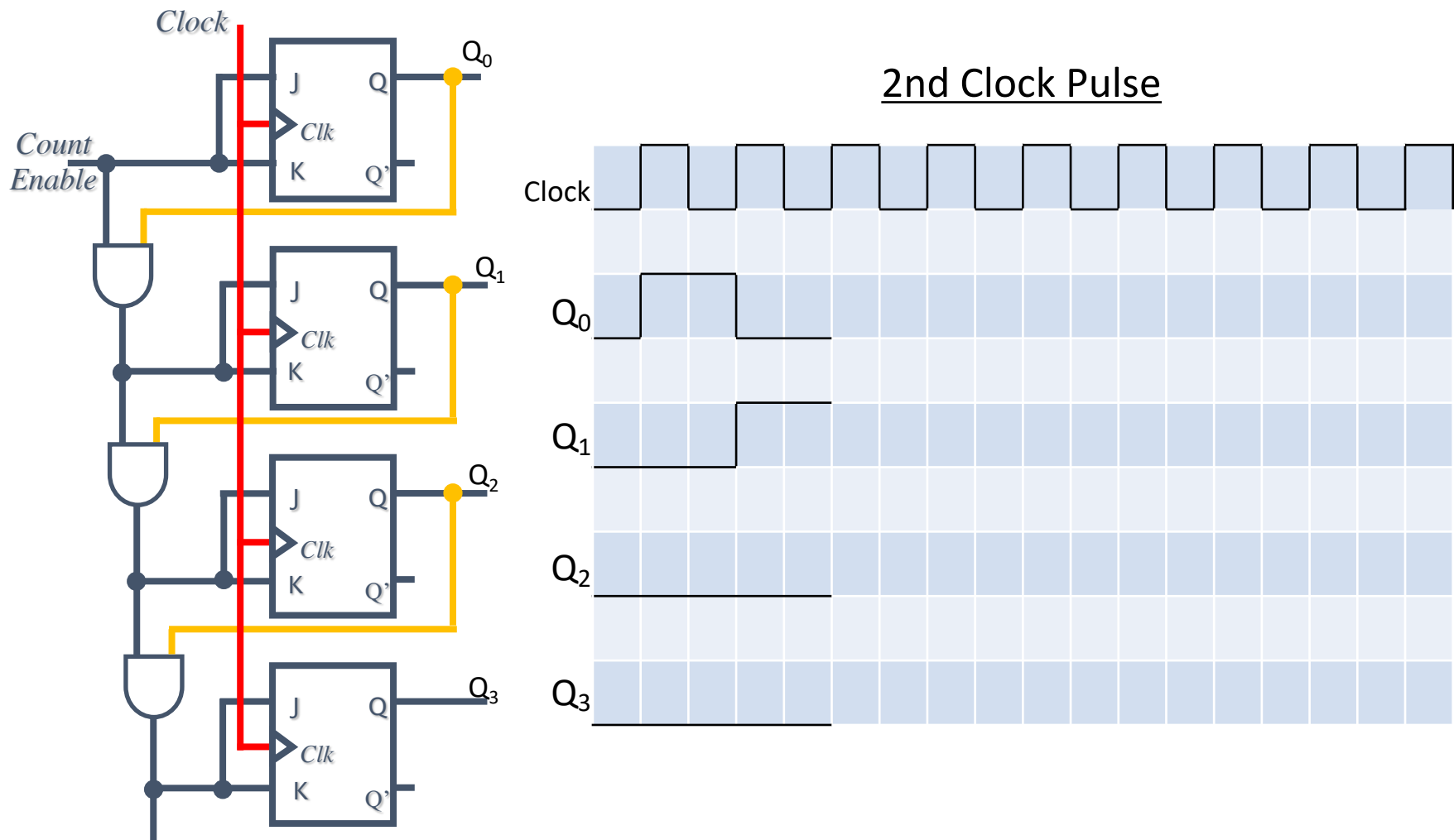
Synchronous Binary Counter (Cont.)



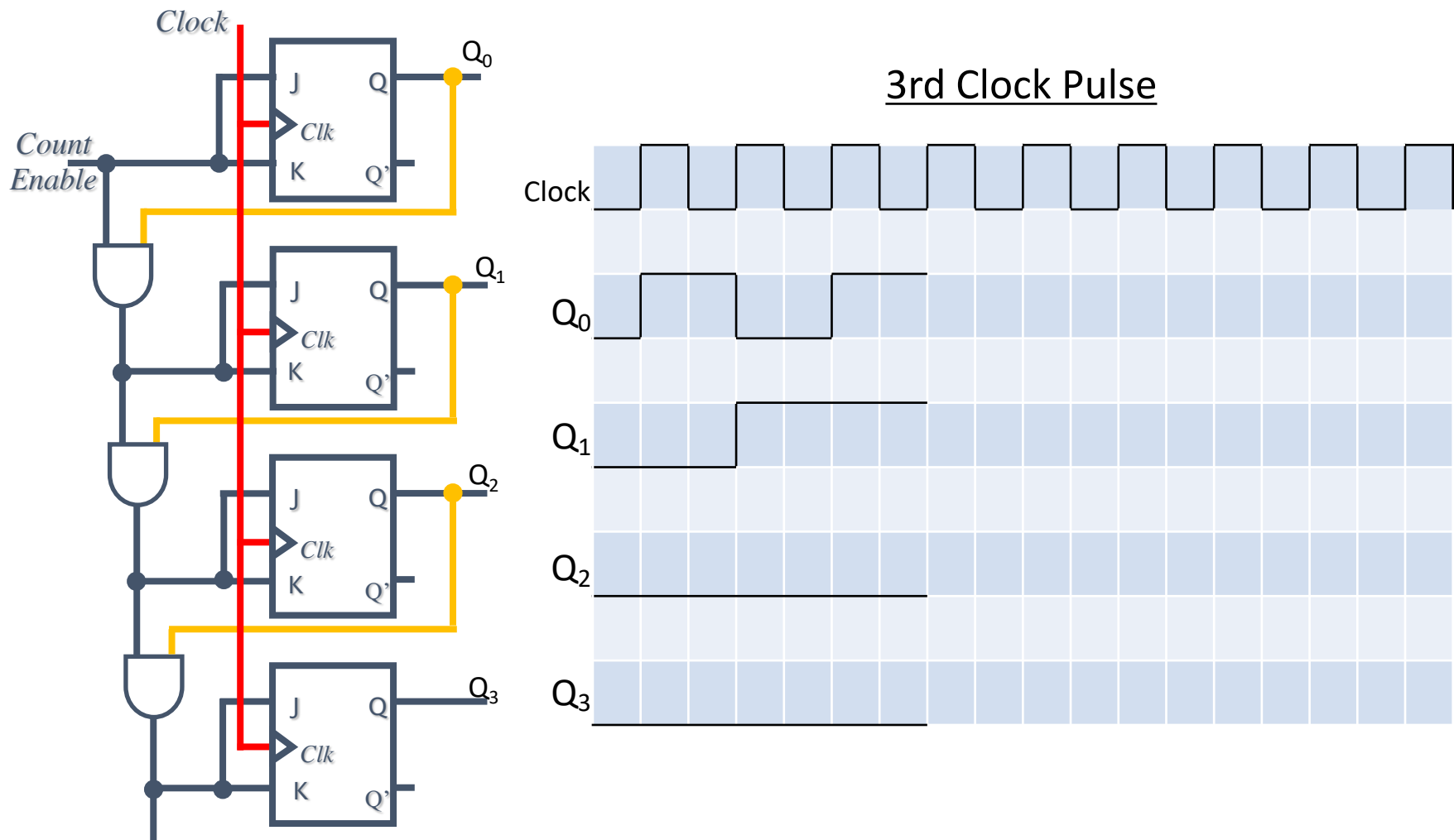
1st Clock Pulse (Positive Edge)



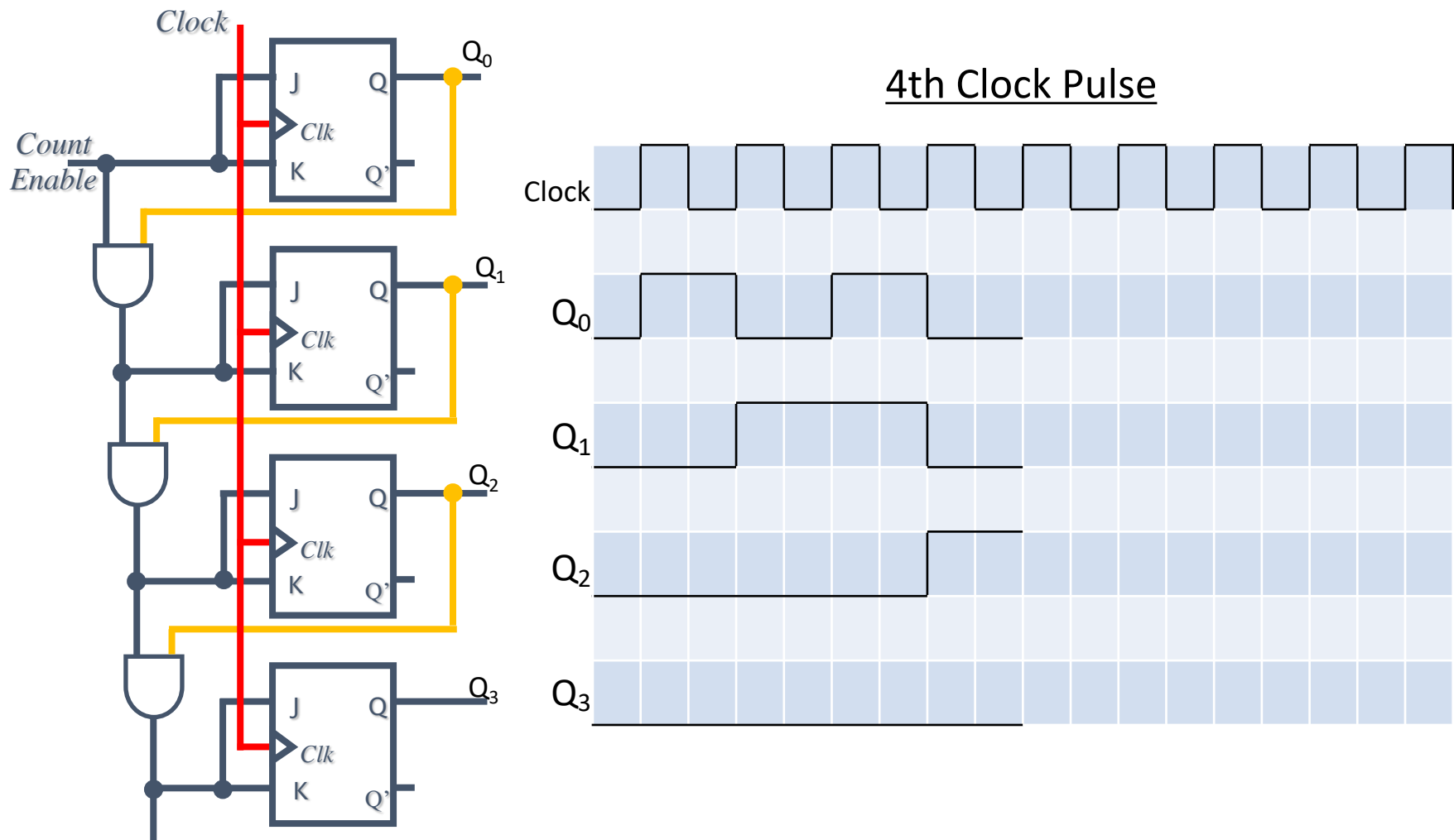
Synchronous Binary Counter (Cont.)



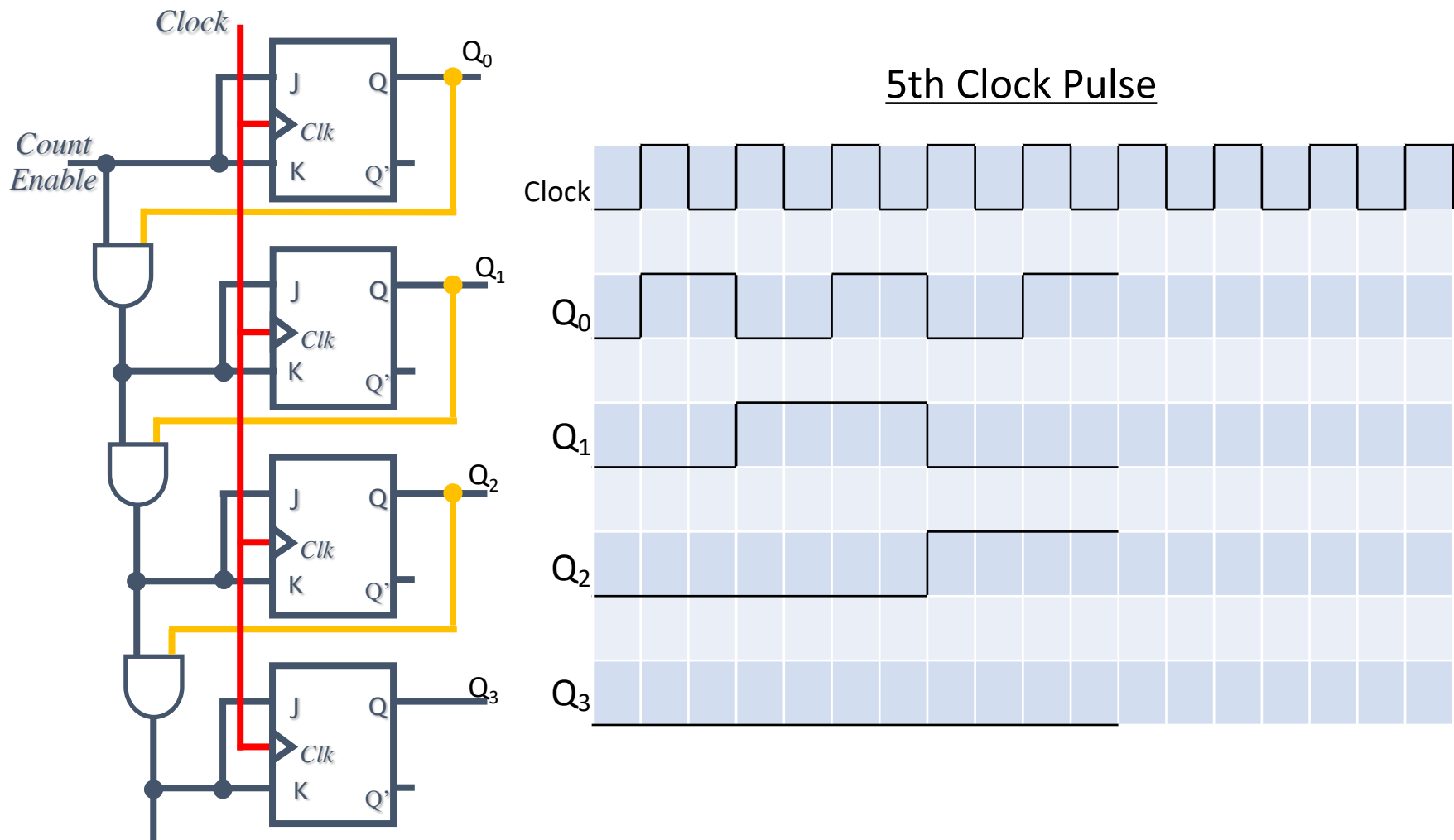
Synchronous Binary Counter (Cont.)



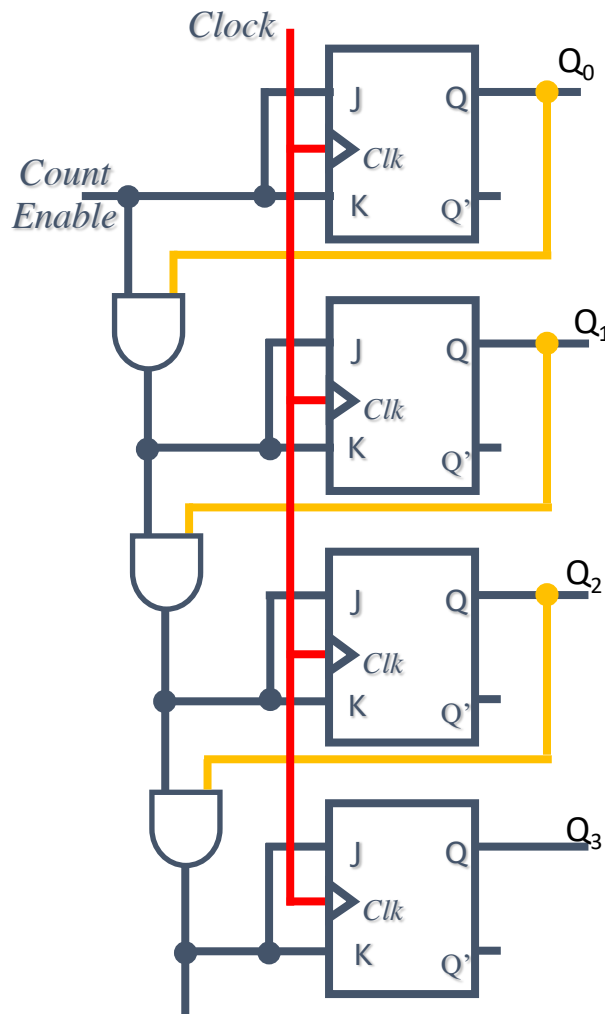
Synchronous Binary Counter (Cont.)



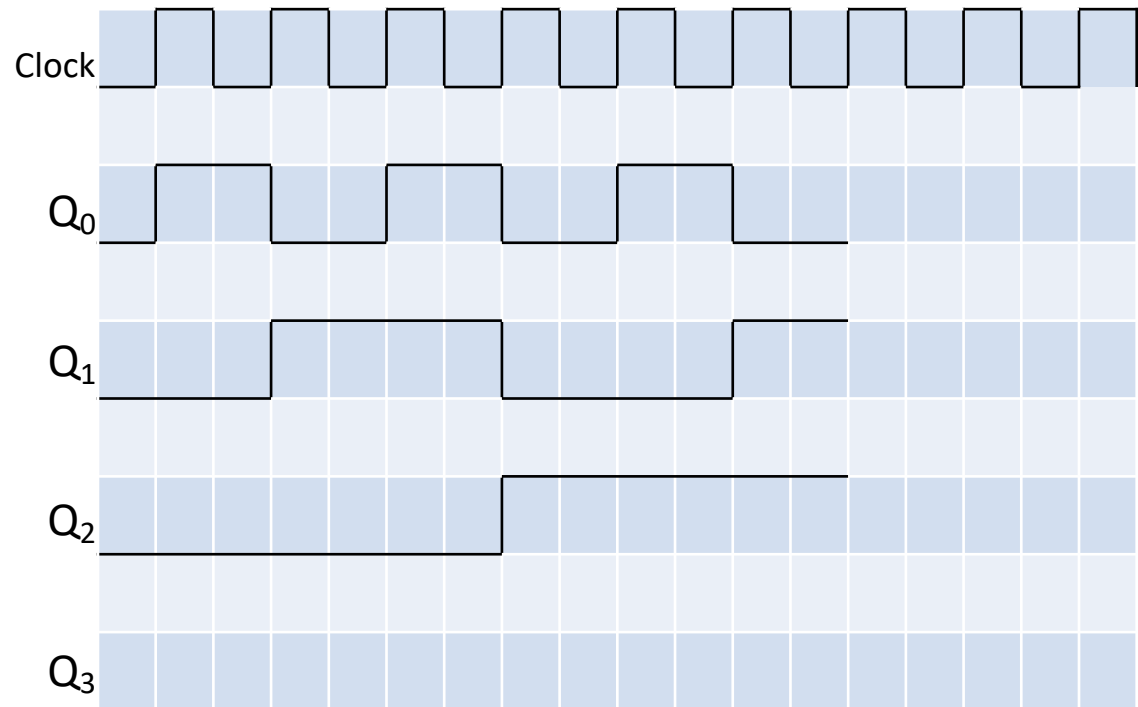
Synchronous Binary Counter (Cont.)



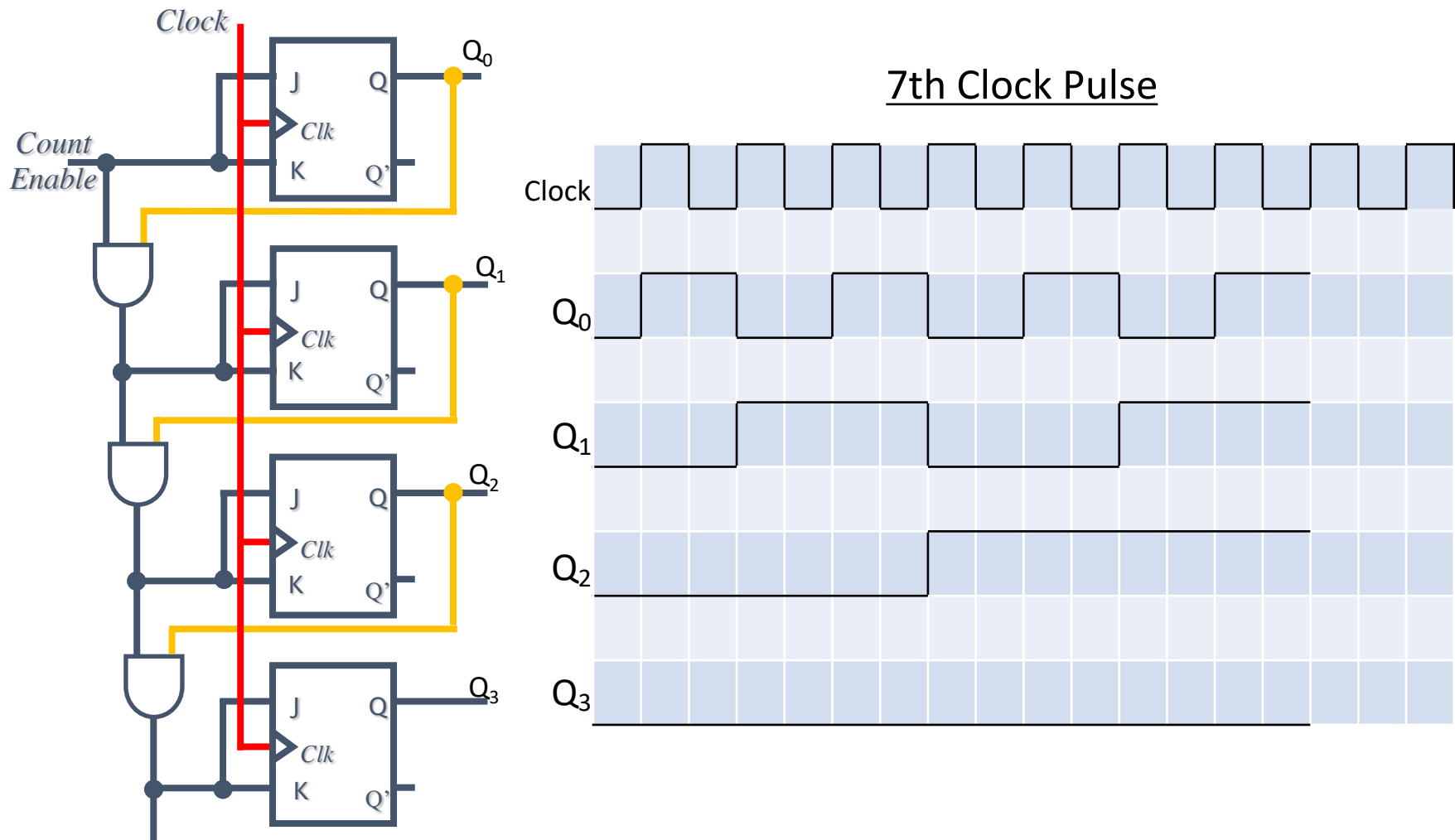
Synchronous Binary Counter (Cont.)



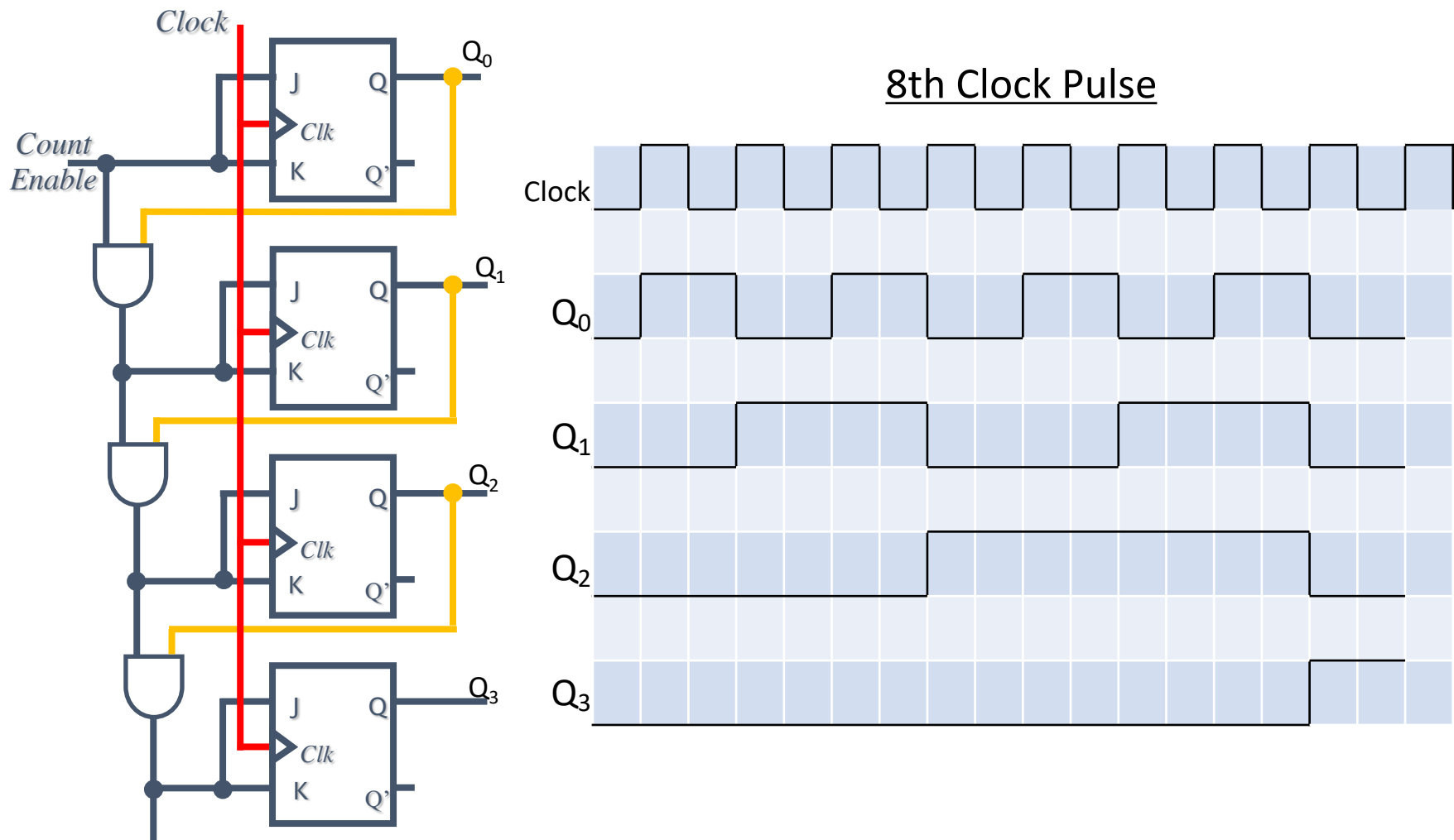
6th Clock Pulse



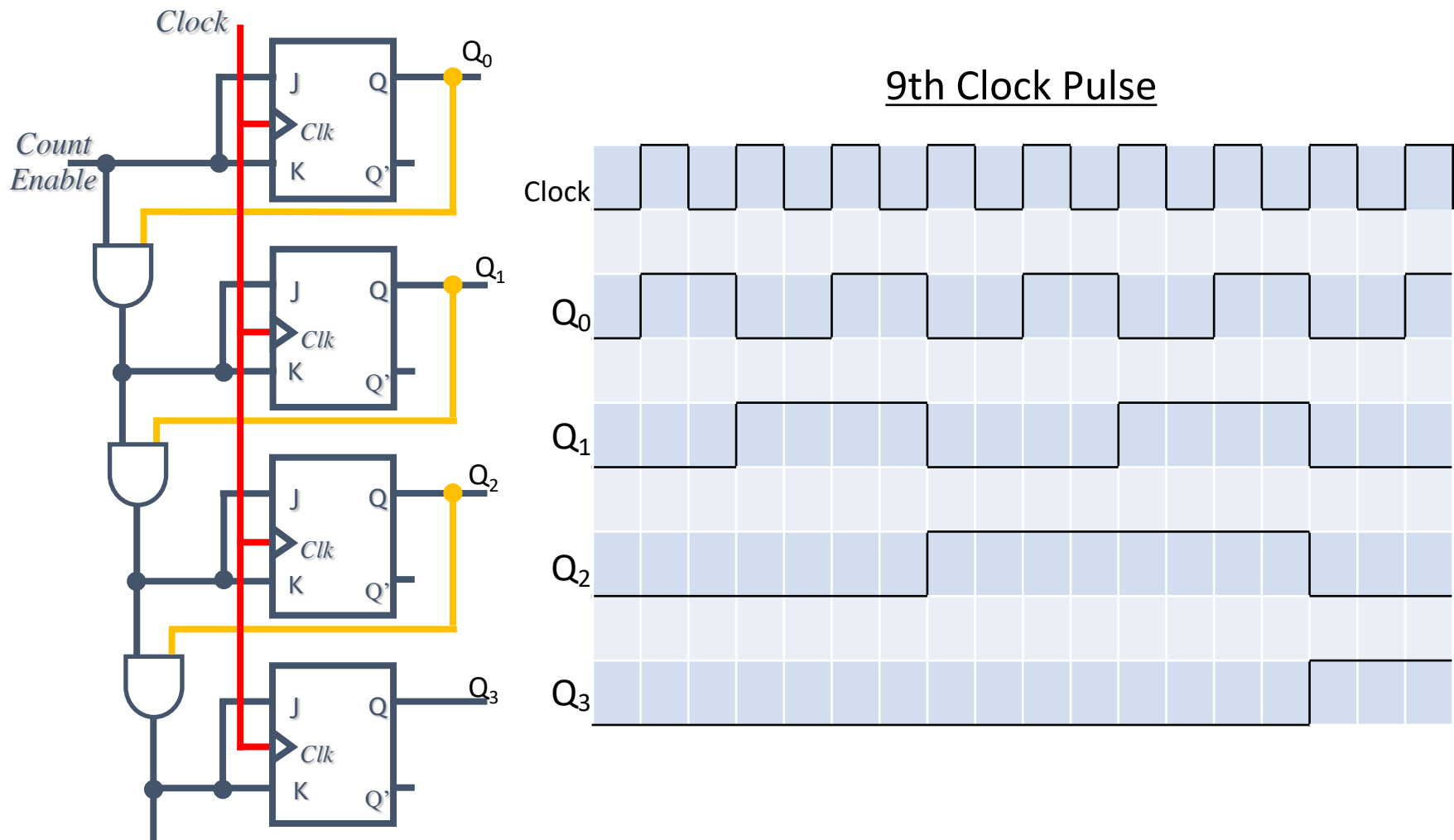
Synchronous Binary Counter. (Cont.)



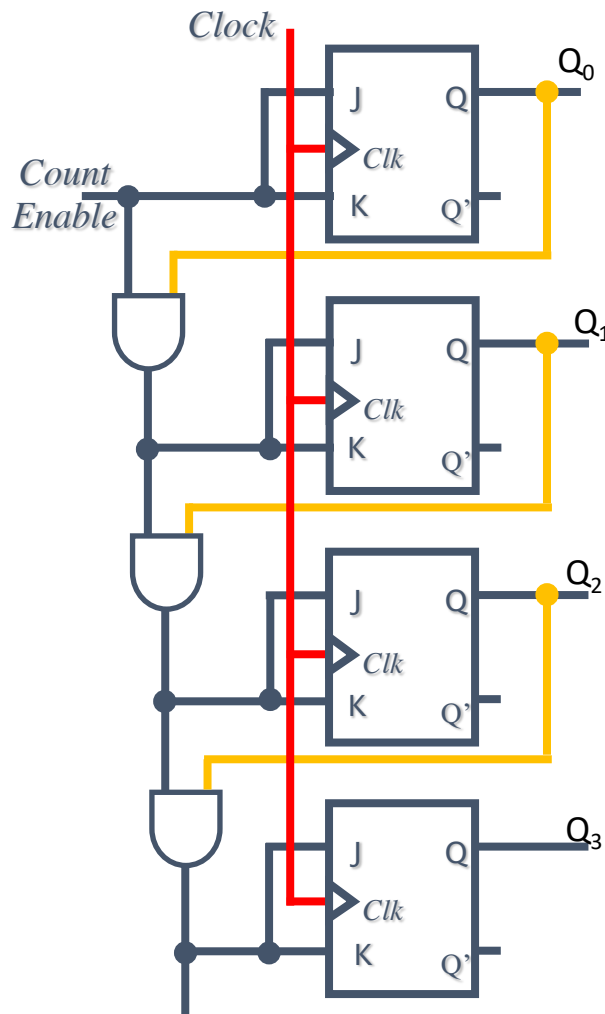
Synchronous Binary Counter (Cont.)



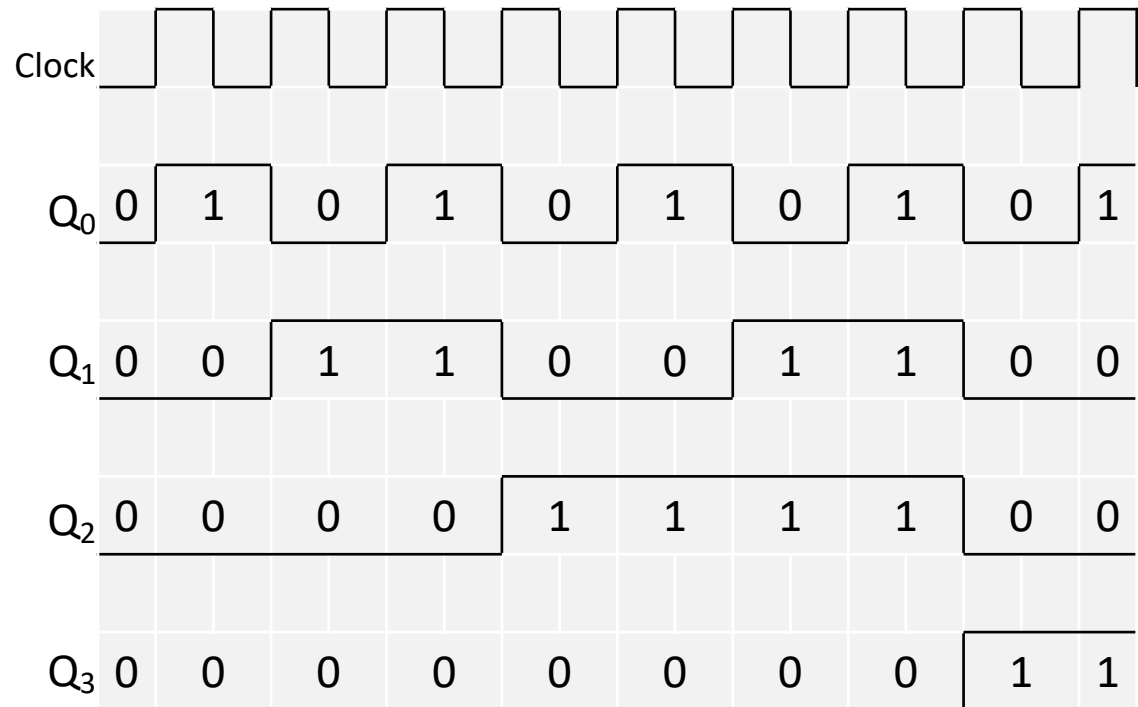
Synchronous Binary Counter. (Cont.)



Synchronous Binary Counter. (Cont.)



9th Clock Pulse



External References

- <http://users.cis.fiu.edu/~prabakar/cda4101/Common/notes/lecture09.html>

Thank You..!
